Lua @ MFP | CIn - UFPE

Sumário

Data Structures

Dsu

```
    /* DSU (Disjoint Set Union / Union-Find)
     * Mantem conjuntos disjuntos com uniao e busca eficiente.
     * Complexidade: O(a(N)); a e a inversa de Ackermann.
     * Memoria: O(N)
     * Metodos:
     * - find(i): Retorna o representante do conjunto de i.
     * - join(i, j): Une os conjuntos de i e j, retorna true se uniu.
     * - same(i, j): Verifica se i e j estao no mesmo conjunto.
     * - size(i): Retorna o tamanho do conjunto de i.
     */
D56 struct DSU {
1A8   int n;
923   vector<int> parent;
1DF   vector<int> sz;
75E   int num_sets;

87D   DSU(int n) : n(n), parent(n), sz(n, 1), num_sets(n) {
4D6     iota(parent.begin(), parent.end(), 0);
3FC   }

C42   int find(int i) {
331     if (parent[i] == i) return i;
565     return parent[i] = find(parent[i]);
72C   }

D58   bool join(int i, int j) {
477     i = find(i), j = find(j);
41E     if (i != j) {
4C2       if (sz[i] < sz[j]) swap(i, j);
1F0       parent[j] = i;
529       sz[i] += sz[j];
CA8       num_sets--;
8A6       return true;
20F     }
D1F     return false;
30C   }
00C   int size(int i) { return sz[find(i)]; }
DA3   bool same(int i, int j) { return find(i) == find(j); }
EF5 };
```

Dsu Rollback

```
    /* DSU com Rollback (Union by Size, sem path compression)
     * Complexidade: O(log N) para find, join e rollback.
     * Memoria: O(N + calls a join)
     * Metodos:
     * - find(i):    raiz do conjunto de i.
     * - join(i, j): une os conjuntos; sempre empilha no historico
     *              (mesmo se ja estavam unidos). Retorna true se uniu.
     * - rollback(): desfaz o ultimo join (empilhado ou fantasma).
     * - same(i, j): verifica se i e j estao no mesmo conjunto.
     * - size(i):    tamanho do conjunto de i.
     *
     * IMPORTANTE: rollback() deve ser chamado uma vez por join(),
     * independente do retorno — join() sempre empilha no historico.
     */
DE6 struct DSURollback {
ADC   int n, num_sets;
89E   vector<int> parent, sz;
AD4   vector<pair<int,int>> history;

A31   DSURollback(int n) : n(n), num_sets(n), parent(n), sz(n, 1) {
4D6     iota(parent.begin(), parent.end(), 0);
F06   }

C42   int find(int i) {
7B9     while (parent[i] != i) i = parent[i];
D9A     return i;
4BA   }

D58   bool join(int i, int j) {
650     i = find(i); j = find(j);
426     if (i == j) { history.push_back({-1, -1}); return false; }
4C2     if (sz[i] < sz[j]) swap(i, j);
1F0     parent[j] = i;
529     sz[i] += sz[j];
CA8     num_sets--;
0E6     history.push_back({j, i});
8A6     return true;
480   }

5CF   void rollback() {
F07     auto [child, p] = history.back();
7D1     history.pop_back();
D1B     if (child != -1) {
B95       sz[p] -= sz[child];
B1B       parent[child] = child;
F4E       num_sets++;
708     }
0D9   }

00C   int size(int i) { return sz[find(i)]; }
DA3   bool same(int i, int j) { return find(i) == find(j); }
FA9 };
```

Mo

```
    /* Mo's Algorithm — O((N + Q) * sqrt(N))
     * queries: vetor de {l, r} 0-indexado, fechado.
     * add(i), rem(i): adicionam/removem o elemento i da janela.
     * ans(qi): registra resposta para a query de indice qi.
     *
     * Exemplo:
     *   vector<int> ans(q);
     *   Mo mo(n, queries);
     *   mo.run(
     *       [&](int i) { ... },
     *       [&](int i) { ... },
     *       [&](int qi) { ans[qi] = ...; }
     *   );
```

```
    */
0DF struct Mo {
7F4   int n, block;
670   struct Query {
738     int l, r, idx;
304   };
240   vector<Query> queries;

8CA   Mo(int n, vector<pair<int, int>> &qs)
10A       : n(n), block(max(1, (int)sqrt(n))) {
D26     for (int i = 0; i < (int)qs.size(); i++)
9AA       queries.push_back({qs[i].first, qs[i].second, i});
F94     sort(queries.begin(), queries.end(),
724          [&](const Query &a, const Query &b) {
ED6            int ba = a.l / block, bb = b.l / block;
A2D            if (ba != bb) return ba < bb;
FDF            return (ba & 1) ? a.r > b.r : a.r < b.r;
FC3          });
8F7   }

D5A   template <typename Add, typename Rem, typename Ans>
743   void run(Add add, Rem rem, Ans ans) {
73A     int cur_l = 0, cur_r = -1;
FFE     for (auto &[l, r, idx] : queries) {
90C       while (cur_r < r) add(++cur_r);
6A3       while (cur_l > l) add(--cur_l);
1B1       while (cur_r > r) rem(cur_r--);
E02       while (cur_l < l) rem(cur_l++);
414       ans(idx);
5B1     }
BB8   }
1FF };

    /* HilbertMo — Mo com ordenacao pela curva de Hilbert
     * Mesma interface que Mo, mas com constante menor na pratica.
     */

CD0 struct HilbertMo {
670   struct Query {
738     int l, r, idx;
BFE     ll ord;
C8D   };
240   vector<Query> queries;

D16   static ll hilbert(int x, int y, int n) {
B72     ll d = 0;
F8A     for (int s = n >> 1; s; s >>= 1) {
7B9       int rx = (x & s) > 0, ry = (y & s) > 0;
71B       d += (ll)s * s * ((3 * rx) ^ ry);
069       if (!ry) {
1C6         if (rx) {
731           x = s - 1 - x;
D41           y = s - 1 - y;
1FD         }
9DD         swap(x, y);
FFF       }
0D1     }
BE2     return d;
416   }

E76   HilbertMo(int n, vector<pair<int, int>> &qs) {
ABE     int hn = 1;
FCC     while (hn < n) hn <<= 1;
D26     for (int i = 0; i < (int)qs.size(); i++)
082       queries.push_back(
A55         {qs[i].first, qs[i].second, i,
979          hilbert(qs[i].first, qs[i].second, hn)});
F94     sort(queries.begin(), queries.end(),
B18          [](const Query &a, const Query &b) {
73A            return a.ord < b.ord;
A3A          });
0D6   }

D5A   template <typename Add, typename Rem, typename Ans>
743   void run(Add add, Rem rem, Ans ans) {
73A     int cur_l = 0, cur_r = -1;
277     for (auto &[l, r, idx, ord] : queries) {
90C       while (cur_r < r) add(++cur_r);
6A3       while (cur_l > l) add(--cur_l);
1B1       while (cur_r > r) rem(cur_r--);
E02       while (cur_l < l) rem(cur_l++);
414       ans(idx);
073     }
DC2   }
1AD };
```

Parallel Binary Search

```
    /* Parallel Binary Search — O((N + Q) log Q * op)
     * Para cada entidade em [0, N), encontra o primeiro evento
     * em [0, Q) apos o qual uma condicao monotona e satisfeita.
     * Requer que a condicao seja monotona: uma vez satisfeita,
     * permanece.
     *
     * apl(i): aplica o evento i na estrutura de dados.
     * rlb(i): desfaz o evento i da estrutura de dados.
     * chk(i): retorna true se a entidade i satisfaz a
     *         condicao dado o estado atual da estrutura.
     *
     * ans[i] = primeiro evento que satisfaz, ou -1 se nunca.
     *
     * Exemplo (soma dos setores de i >= limite[i]):
     *   ParallelBS pbs(n, q);
     *   pbs.run(
     *       [&](int i) { seg.update(l[i], r[i], {x[i]}); },
     *       [&](int i) { seg.update(l[i], r[i], {-x[i]}); },
     *       [&](int idx) {
     *           ll sum = 0;
     *           for (int s : sectors[idx]) {
     *               sum += seg.query(s, s).val;
     *               if (sum >= lim[idx]) return true;
     *           }
     *           return false;
     *       }
     *   );
     *   for (int i = 0; i < n; i++)
     *       cout << (pbs.ans[i] != -1 ? pbs.ans[i]+1 : -1);
     */
540 struct ParallelBS {
3B9     int n, q;
DAB     vector<int> ans;

DC7     ParallelBS(int n, int q) : n(n), q(q), ans(n, -1) {}

8BC     template <typename Apl, typename Rlb, typename Chk>
708     void run(Apl apl, Rlb rlb, Chk chk) {
56A         vector<int> actv(n);
A0A         iota(actv.begin(), actv.end(), 0);
483         solve(0, q - 1, actv, apl, rlb, chk);
41B     }

BF2   private:
8BC     template <typename Apl, typename Rlb, typename Chk>
EF4     void solve(int l, int r, vector<int>& actv,
9A3                Apl apl, Rlb rlb, Chk chk) {
658         if (actv.empty()) return;

893         if (l == r) {
A2E             apl(l);
E01             for (auto x : actv) if (chk(x)) ans[x] = l;
623             rlb(l);
505             return;
78E         }

318         int mid = l + (r - l) / 2;
9EA         for (int i = l; i <= mid; i++) apl(i);

37F         vector<int> pass, not_pass;
3FE         for (auto x : actv)
C9B             (chk(x) ? pass : not_pass).push_back(x);

9F1         solve(mid + 1, r, not_pass, apl, rlb, chk);
A2B         for (int i = l; i <= mid; i++) rlb(i);
50E         solve(l, mid, pass, apl, rlb, chk);
3FC     }
616 };
```

Fenwick Tree

```
    /* Fenwick Tree (Point Update, Prefix Query)
     * Realiza atualizacoes pontuais e consultas de prefixo.
     * Complexidade: O(log N) para update e query.
     * Memoria: O(N)
     * Requisitos:
     * - NODE: operator+=, operator- e construtor default.
     */

    /* --- Exemplo de NODE (Soma) ---
    struct Node {
        ll val;
        Node(ll v = 0) : val(v) {}
        void operator+=(const Node& other) { val += other.val; }
        Node operator-(const Node& other) const {
            return Node(val - other.val);
        }
    };
    */
8A0 template <typename NODE>
0F1 struct FenwickTree {
060   int N;
9B8   vector<NODE> bit;

5D2   FenwickTree(int n) : N(n), bit(n + 1) {}

D6B   void update(int idx, NODE v) {
B2F     for (idx++; idx <= N; idx += idx & -idx) {
046       bit[idx] += v;
23E     }
9DA   }

FCA   NODE query(int idx) { // [0, idx]
ADA     NODE res;
159     for (idx++; idx > 0; idx -= idx & -idx) {
8E6       res += bit[idx];
A38     }
B50     return res;
C00   }

762   NODE query(int l, int r) { // [l, r]
7C3     if (l > r) return NODE();
A4E     return query(r) - query(l - 1);
821   }
134 };
```

Fenwick Tree 2D

```
    /* Fenwick Tree 2D (Point Update, Rectangle Query)
     * Atualizacoes pontuais e consultas de prefixo em matrizes.
     * Complexidade: O(log N * log M) para update e query.
     * Memoria: O(N * M)
     * Requisitos:
     * - NODE: operator+=, operator- e construtor default.
     */

    /* --- Exemplo de NODE (Soma) ---
    struct Node {
        ll val;
        Node(ll v = 0) : val(v) {}
        void operator+=(const Node& other) { val += other.val; }
        Node operator-(const Node& other) const {
            return Node(val - other.val);
        }
    };
    */
8A0 template <typename NODE>
2E6 struct FenwickTree2D {
610   int N, M;
803   vector<vector<NODE>> bit;
```

```
265   FenwickTree2D(int n, int m)
B1E       : N(n), M(m), bit(n + 1, vector<NODE>(m + 1)) {}

274   void update(int r, int c, NODE v) { // point update
3BA     for (int i = r + 1; i <= N; i += i & -i) {
786       for (int j = c + 1; j <= M; j += j & -j) {
9ED         bit[i][j] += v;
95F       }
A1F     }
BF9   }

82E   NODE query(int r, int c) { // [(0,0), (r,c)]
ADA     NODE res;
8D9     for (int i = r + 1; i > 0; i -= i & -i) {
83F       for (int j = c + 1; j > 0; j -= j & -j) {
01D         res += bit[i][j];
CB5       }
E20     }
B50     return res;
943   }

      // Retangulo fechado [(r1,c1), (r2,c2)].
05F   NODE query(int r1, int c1, int r2, int c2) {
5C4     if (r1 > r2 || c1 > c2) return NODE();
10A     return query(r2, c2) - query(r1 - 1, c2) -
7F5            query(r2, c1 - 1) + query(r1 - 1, c1 - 1);
0E7   }
FF7 };
```

Dynamic Fenwick Tree 2D

```
    /* Fenwick Tree 2D Sparse (Point Update, Rectangle Query)
     * Suporta coordenadas ate 10^9 usando compressao interna.
     * Complexidade: Build O(P log P), Update/Query O(log^2 P).
     * Memoria: O(P log P).
     * Requisitos:
     * - NODE: operator+=, operator-, operator+ e construtor default.
     * - Offline: passe todos os pontos de interesse no build.
     *
     * Creditos: Adaptado de TFG (Thiago Ferreira Goncalves)
     * Fonte: github.com/tfg50/Competitive-Programming
     */

    /* --- Exemplo de NODE (Soma com Inclusao-Exclusao) ---
    struct Node {
        ll val;
        Node(ll v = 0) : val(v) {}

        void operator+=(const Node& other) {
            val += other.val;
        }

        Node operator-(const Node& other) const {
            return Node(val - other.val);
        }

        Node operator+(const Node& other) const {
            return Node(val + other.val);
        }
    };
    */
8A0 template <typename NODE>
222 struct FenwickTree2DSparse {
6C1   vector<int> ord;
803   vector<vector<NODE>> bit;
E76   vector<vector<int>> coord;

BEF   FenwickTree2DSparse(vector<pii> pts) {
CD7     sort(pts.begin(), pts.end());
5A8     for (auto [x, y] : pts) {
DC4       if (ord.empty() || x != ord.back()) ord.push_back(x);
ABA     }

261     bit.resize(ord.size() + 1);
3EB     coord.resize(ord.size() + 1);

A0A     sort(pts.begin(), pts.end(), [](const pii &a, const pii &b) {
463       return a.second < b.second;
19C     });

5A8     for (auto [x, y] : pts) {
9DE       for (int i = get_x(x); i < bit.size(); i += i & -i) {
3E1         if (coord[i].empty() || coord[i].back() != y)
739           coord[i].push_back(y);
4F2       }
B39     }

8C2     for (int i = 0; i < bit.size(); i++)
F3E       bit[i].assign(coord[i].size() + 1, NODE());
D9D   }

7E8   int get_x(int x) {
2C8     return upper_bound(ord.begin(), ord.end(), x) - ord.begin();
95C   }

D5E   int get_y(int i, int y) {
E18     auto &c = coord[i];
BBD     return upper_bound(c.begin(), c.end(), y) - c.begin();
418   }

5CF   void update(int x, int y, NODE v) {
9DE     for (int i = get_x(x); i < bit.size(); i += i & -i) {
813       for (int j = get_y(i, y); j < bit[i].size(); j += j & -j) {
9ED         bit[i][j] += v;
34E       }
F86     }
390   }

84C   NODE query(int x, int y) {
ADA     NODE res;
FAD     for (int i = get_x(x); i > 0; i -= i & -i) {
263       for (int j = get_y(i, y); j > 0; j -= j & -j) {
01D         res += bit[i][j];
359       }
92F     }
B50     return res;
7C4   }

FCA   NODE query(int x1, int y1, int x2, int y2) {
276     return query(x2, y2) - query(x1 - 1, y2) -
12D            query(x2, y1 - 1) + query(x1 - 1, y1 - 1);
944   }
B5F };
```

Sparse Table

```
    /* Sparse Table (Static RMQ)
     * Realiza consultas em range em tempo constante.
     * Complexidade: O(N log N) no build, O(1) na query.
     * Memoria: O(N log N)
     * Requisitos:
     * - NODE deve ter: static merge(const NODE&, const NODE&) e
     * construtor.
     * - A operacao DEVE ser idempotente: f(a, a) = a.
     */

    /* --- Exemplo de NODE (Indice do Minimo) ---
    struct Node {
        int val, pos;
        Node(int v = 2e9, int p = -1) : val(v), pos(p) {}

        static Node merge(const Node& l, const Node& r) {
            return l.val <= r.val ? l : r;
        }
    };

    // Uso: SparseTable<Node> st(v) onde v e vector<Node>
    // for (int i = 0; i < n; i++) {
    //   cin >> x; v[i] = Node(x, i);
    // }
    */
8A0 template <typename NODE>
7E9 struct SparseTable {
5F0   int N, K;
6F5   vector<vector<NODE>> st;
4A7   vector<int> lg;

67A   template <typename T>
E4A   SparseTable(const vector<T> &v) : N(v.size()) {
A77     K = (N > 0) ? __lg(N) : 0;
3A9     st.assign(K + 1, vector<NODE>(N));
641     lg.assign(N + 1, 0);
BB5     for (int i = 2; i <= N; i++) lg[i] = lg[i / 2] + 1;
DDD     for (int i = 0; i < N; i++) st[0][i] = NODE(v[i]);
2CE     for (int j = 1; j <= K; j++)
285       for (int i = 0; i + (1 << j) <= N; i++)
4E1         st[j][i] = NODE::merge(st[j - 1][i],
A64                                st[j - 1][i + (1 << (j - 1))]);
CE2   }

762   NODE query(int l, int r) {
2C3     int j = lg[r - l + 1];
8F2     return NODE::merge(st[j][l], st[j][r - (1 << j) + 1]);
843   }
231 };
```

Disjoint Sparse Table

```
    /* Disjoint Sparse Table (Static Range Query)
     * Query O(1) para qualquer operacao associativa.
     * Complexidade: O(N log N) no build, O(1) na query.
     * Memoria: O(N log N)
     * Requisitos:
     * - NODE deve ter: static merge(const NODE&, const NODE&) e
     * construtor.
     * - Operacao deve ser ASSOCIATIVA: f(f(a,b),c) =
     * f(a,f(b,c)).
     */

    /* --- Exemplo de NODE (Soma em Range) ---
    struct Node {
        ll val;
        Node(ll v = 0) : val(v) {}
        static Node merge(const Node& l, const Node& r) {
            return Node(l.val + r.val);
        }
    };
    */
8A0 template <typename NODE>
906 struct DisjointSparseTable {
5F0   int N, K;
6F5   vector<vector<NODE>> st;

67A   template <typename T>
68E   DisjointSparseTable(const vector<T> &v) : N(v.size()) {
3AA     if (N == 0) return;
475     K = __lg(2 * N - 1);
C78     st.assign(K, vector<NODE>(1 << K));

DDD     for (int i = 0; i < N; i++) st[0][i] = NODE(v[i]);

C54     for (int j = 1; j < K; j++) {
1C0       int len = 1 << j;
C2C       for (int m = len; m < (1 << K); m += 2 * len) {
A34         if (m < N) st[j][m] = st[0][m];
6D6         for (int i = m + 1; i < min(N, m + len); i++)
0EA           st[j][i] = NODE::merge(st[j][i - 1], st[0][i]);

C2E         if (m - 1 < N) st[j][m - 1] = st[0][m - 1];
226         for (int i = m - 2; i >= m - len; i--)
604           st[j][i] = NODE::merge(st[0][i], st[j][i + 1]);
643       }
3F0     }
4D7   }

762   NODE query(int l, int r) {
434     if (l == r) return st[0][l];
B89     int j = __lg(l ^ r);
894     return NODE::merge(st[j][l], st[j][r]);
33A   }
04C };
```

Segment Tree

```
    /* Segment Tree (Point Update, Range Query)
     * Complexidade: O(log N) para update e query.
     * Memoria: O(4*N)
     * Requisitos:
     * - NODE deve ter: static merge(L, R), apply(V), e
     * construtor identidade.
     */
    /* --- Exemplo de NODE (Soma com Update de Substituicao) ---
    struct Node {
        ll val = 0;
        Node(ll v = 0) : val(v) {}
        static Node merge(const Node& l, const Node& r) {
            return Node(l.val + r.val);
        }
        // Para Point Set: val = v;
        // Para Point Add: val += v;
        void apply(ll v) {
            val = v;
        }
    };
    */
8A0 template <typename NODE>
383 struct SegTree {
060   int N;
B1C   vector<NODE> seg;
FDE   explicit SegTree(int n) : N(n), seg(4 * n) {}
67A   template <typename T>
5DE   SegTree(const vector<T> &v) : SegTree((int)v.size()) {
231     build(1, 0, N - 1, v);
73D   }
67A   template <typename T>
5B9   void build(int no, int l, int r, const vector<T> &v) {
893     if (l == r) {
C3E       seg[no] = NODE(v[l]);
505       return;
B2E     }
27E     int m = (l + r) >> 1;
35E     build(no << 1, l, m, v);
C55     build((no << 1) | 1, m + 1, r, v);
DEB     seg[no] = NODE::merge(seg[no << 1], seg[(no << 1) | 1]);
349   }
637   template <typename V>
AF7   void update(int no, int l, int r, int idx, V val) {
893     if (l == r) {
D88       seg[no].apply(val);
505       return;
EC9     }
27E     int m = (l + r) >> 1;
B73     if (idx <= m) update(no << 1, l, m, idx, val);
99C     else update((no << 1) | 1, m + 1, r, idx, val);
DEB     seg[no] = NODE::merge(seg[no << 1], seg[(no << 1) | 1]);
7BC   }
38D   NODE query(int no, int l, int r, int a, int b) {
2E6     if (b < l || r < a) return NODE();
83F     if (a <= l && r <= b) return seg[no];
27E     int m = (l + r) >> 1;
AFE     return NODE::merge(query(no << 1, l, m, a, b),
074                        query((no << 1) | 1, m + 1, r, a, b));
365   }
637   template <typename V>
864   void update(int idx, V val) {
E1B     update(1, 0, N - 1, idx, val);
22F   }
36F   NODE query(int l, int r) { return query(1, 0, N - 1, l, r); }
0C7 };
```

Lazy Segment Tree

```
    /* Lazy Segment Tree (Range Update, Range Query)
     * Realiza atualizacoes e consultas em range.
     * Complexidade: O(log N) para update e query.
     * Memoria: O(4*N)
     * Requisitos:
     * - NODE deve ter: static merge(L, R), apply(TAG, L, R) e
     *   construtor identidade.
     * - TAG deve ter: compose(TAG) e construtor identidade.
     */
    /* --- Exemplo de NODE e TAG (Soma e Multiplicacao com
     * Modulo) ---
    struct Tag {
        ll mul = 1, add = 0;
        void compose(const Tag& t) {
            add = (add * t.mul + t.add) % MOD;
            mul = (mul * t.mul) % MOD;
        }
    };
    struct Node {
        ll val = 0;
        Node(ll v = 0) : val(v) {}
        static Node merge(const Node& l, const Node& r) {
            return Node((l.val + r.val) % MOD);
        }
        void apply(const Tag& t, int l, int r) {
            val = (val * t.mul + t.add * (r - l + 1)) % MOD;
        }
    };
    */
708 template <typename NODE, typename TAG>
2BA struct LazySegmentTree {
060   int N;
B1C   vector<NODE> seg;
71A   vector<TAG> lazy;
BB8   explicit LazySegmentTree(int n)
63C       : N(n), seg(4 * n), lazy(4 * n) {}
67A   template <typename T>
1BA   LazySegmentTree(const vector<T> &v)
002       : LazySegmentTree((int)v.size()) {
231     build(1, 0, N - 1, v);
9BB   }
67A   template <typename T>
5B9   void build(int no, int l, int r, const vector<T> &v) {
893     if (l == r) {
C3E       seg[no] = NODE(v[l]);
505       return;
B2E     }
27E     int m = (l + r) >> 1;
35E     build(no << 1, l, m, v);
C55     build((no << 1) | 1, m + 1, r, v);
DEB     seg[no] = NODE::merge(seg[no << 1], seg[(no << 1) | 1]);
349   }
F1A   void push(int no, int lo, int hi) {
946     int m = (lo + hi) >> 1;
8F3     int l = no << 1, r = l | 1;
D09     seg[l].apply(lazy[no], lo, m);
9FE     lazy[l].compose(lazy[no]);
204     seg[r].apply(lazy[no], m + 1, hi);
3FC     lazy[r].compose(lazy[no]);
47F     lazy[no] = TAG();
DD1   }
130   void update(int no, int l, int r, int a, int b, const TAG &v) {
2E6     if (b < l || r < a) return;
02B     if (a <= l && r <= b) {
8C7       seg[no].apply(v, l, r);
92C       lazy[no].compose(v);
505       return;
81F     }
FF9     push(no, l, r);
27E     int m = (l + r) >> 1;
9AB     update(no << 1, l, m, a, b, v);
087     update((no << 1) | 1, m + 1, r, a, b, v);
DEB     seg[no] = NODE::merge(seg[no << 1], seg[(no << 1) | 1]);
A1A   }
38D   NODE query(int no, int l, int r, int a, int b) {
2E6     if (b < l || r < a) return NODE();
83F     if (a <= l && r <= b) return seg[no];
FF9     push(no, l, r);
27E     int m = (l + r) >> 1;
AFE     return NODE::merge(query(no << 1, l, m, a, b),
074                        query((no << 1) | 1, m + 1, r, a, b));
80A   }
4E7   void update(int l, int r, const TAG &v) {
E27     update(1, 0, N - 1, l, r, v);
0D7   }
36F   NODE query(int l, int r) { return query(1, 0, N - 1, l, r); }
3CE };
```

Dual Segment Tree

```
    /* Dual Segment Tree (Range Update, Point Query)
     * Realiza atualizacoes em range e consultas pontuais.
     * Complexidade: O(log N) para update e get.
     * Memoria: O(4*N)
     * Requisitos:
     * - TAG deve ter: compose(TAG), apply(T&) e construtor
     * identidade.
     * - T e o tipo do dado nas folhas.
     * Exemplo de uso: DualSegTree<ll, MyTag> st(v);
     */
    /* --- Exemplo de TAG (Soma e Multiplicacao com Modulo) ---
    struct Tag {
        ll mul = 1, add = 0;
        void compose(const Tag& t) {
            mul = (mul * t.mul) % MOD;
            add = (add * t.mul + t.add) % MOD;
        }
        void apply(ll& val) { val = (val * mul + add) % MOD; }
    };
    */
2E5 template <typename T, typename TAG>
9C1 struct DualSegTree {
060   int N;
71A   vector<TAG> lazy;
E4B   vector<T> leaves;
F06   explicit DualSegTree(int n) : N(n), lazy(4 * n), leaves(n) {}
5F6   DualSegTree(const vector<T> &v)
CE6       : N((int)v.size()), lazy(4 * v.size()), leaves(v) {}
78E   void push(int no) {
8A1     int l = no << 1;
C9C     int r = l | 1;
9FE     lazy[l].compose(lazy[no]);
3FC     lazy[r].compose(lazy[no]);
47F     lazy[no] = TAG();
283   }
130   void update(int no, int l, int r, int a, int b, const TAG &v) {
2E6     if (b < l || r < a) return;
02B     if (a <= l && r <= b) {
92C       lazy[no].compose(v);
505       return;
CA0     }
523     push(no);
27E     int m = (l + r) >> 1;
9AB     update(no << 1, l, m, a, b, v);
087     update((no << 1) | 1, m + 1, r, a, b, v);
BEB   }
```

```
629   T get(int no, int l, int r, int idx) {
893     if (l == r) {
4C5       T res = leaves[idx];
2E3       lazy[no].apply(res);
B50       return res;
BE7     }
523     push(no);
27E     int m = (l + r) >> 1;
52E     if (idx <= m) return get(no << 1, l, m, idx);
483     else return get((no << 1) | 1, m + 1, r, idx);
C4A   }
C8C   void update(int l, int r, const TAG &t) {
56D     update(1, 0, N - 1, l, r, t);
A21   }
AE4   T get(int idx) { return get(1, 0, N - 1, idx); }
25C };
```

Segment Tree Beats

```
    /* Segment Tree Beats (Range Update, Range Query)
     * Lazy Seg Tree com condicoes de parada e aplicacao de tag.
     * Complexidade amortizada: O(N log² N) por sequencia de updates.
     * Memoria: O(4*N)
     * Requisitos:
     * - NODE deve ter: static merge(L, R), apply(TAG, L, R),
     *   break_condition(TAG) e tag_condition(TAG),
     *   e construtor identidade.
     * - TAG deve ter: compose(TAG) e construtor identidade.
     *
     * break_condition(tag): ignora update ja satisfeito (mx <= v).
     * tag_condition(tag): aplica sem descer (mx2 < v <= mx).
     */
    /* --- Exemplo de NODE e TAG (chmin em range + query de
     * soma/max) ---
    struct Tag {
        ll val = LLONG_MAX; // identidade: nao faz nada
        void compose(const Tag& t) {
            val = min(val, t.val);
        }
    };
    struct Node {
        ll sum, mx, mx2;
        int cnt_mx, sz;
        Node()
            : sum(0), mx(LLONG_MIN), mx2(LLONG_MIN),
              cnt_mx(0), sz(0) {}
        Node(ll v)
            : sum(v), mx(v), mx2(LLONG_MIN), cnt_mx(1),
              sz(1) {}
        static Node merge(const Node& l, const Node& r) {
            Node res;
            res.sz = l.sz + r.sz;
            res.sum = l.sum + r.sum;
            if (l.mx == r.mx) {
                res.mx = l.mx; res.cnt_mx = l.cnt_mx + r.cnt_mx;
                res.mx2 = max(l.mx2, r.mx2);
            } else if (l.mx > r.mx) {
                res.mx = l.mx; res.cnt_mx = l.cnt_mx;
                res.mx2 = max(l.mx2, r.mx);
            } else {
                res.mx = r.mx; res.cnt_mx = r.cnt_mx;
                res.mx2 = max(l.mx, r.mx2);
            }
            return res;
        }
        // aplica a tag quando tag_condition e verdadeiro
        void apply(const Tag& t, int l, int r) {
            sum -= (ll)(mx - t.val) * cnt_mx;
            mx = t.val;
        }
        // ignora o update completamente (no ja satisfaz)
        bool break_condition(const Tag& t) const {
            return mx <= t.val;
        }
        // pode aplicar a tag diretamente sem descer
        bool tag_condition(const Tag& t) const {
            return mx2 < t.val;
        }
    };
    */
708 template <typename NODE, typename TAG>
BC8 struct SegTreeBeats {
060   int N;
B1C   vector<NODE> seg;
71A   vector<TAG> lazy;
249   explicit SegTreeBeats(int n) : N(n), seg(4 * n), lazy(4 * n) {}
67A   template <typename T>
47B   SegTreeBeats(const vector<T> &v)
001       : SegTreeBeats((int)v.size()) {
231     build(1, 0, N - 1, v);
1EF   }
67A   template <typename T>
5B9   void build(int no, int l, int r, const vector<T> &v) {
893     if (l == r) {
C3E       seg[no] = NODE(v[l]);
505       return;
B2E     }
27E     int m = (l + r) >> 1;
35E     build(no << 1, l, m, v);
C55     build((no << 1) | 1, m + 1, r, v);
DEB     seg[no] = NODE::merge(seg[no << 1], seg[(no << 1) | 1]);
349   }
F1A   void push(int no, int lo, int hi) {
946     int m = (lo + hi) >> 1;
8F3     int l = no << 1, r = l | 1;
D09     seg[l].apply(lazy[no], lo, m);
9FE     lazy[l].compose(lazy[no]);
204     seg[r].apply(lazy[no], m + 1, hi);
3FC     lazy[r].compose(lazy[no]);
47F     lazy[no] = TAG();
DD1   }
130   void update(int no, int l, int r, int a, int b, const TAG &v) {
FBB     if (b < l || r < a || seg[no].break_condition(v)) return;
0A9     if (l == r) return seg[no].apply(v, l, r), void();
40D     if (a <= l && r <= b && seg[no].tag_condition(v)) {
8C7       seg[no].apply(v, l, r);
92C       lazy[no].compose(v);
505       return;
2B2     }
FF9     push(no, l, r);
27E     int m = (l + r) >> 1;
9AB     update(no << 1, l, m, a, b, v);
087     update((no << 1) | 1, m + 1, r, a, b, v);
DEB     seg[no] = NODE::merge(seg[no << 1], seg[(no << 1) | 1]);
F37   }
38D   NODE query(int no, int l, int r, int a, int b) {
2E6     if (b < l || r < a) return NODE();
83F     if (a <= l && r <= b) return seg[no];
FF9     push(no, l, r);
27E     int m = (l + r) >> 1;
AFE     return NODE::merge(query(no << 1, l, m, a, b),
074                        query((no << 1) | 1, m + 1, r, a, b));
80A   }
4E7   void update(int l, int r, const TAG &v) {
E27     update(1, 0, N - 1, l, r, v);
0D7   }
36F   NODE query(int l, int r) { return query(1, 0, N - 1, l, r); }
FCC };
```

Dynamic Segment Tree

```
    /* Dynamic Segment Tree + Lazy (Range Update, Range Query)
     * Utilizada para intervalos gigantes (ex: 0 a 10^18).
     * Complexidade: O(log N) por operacao.
     * Memoria: O(Q log N) onde Q e o numero de operacoes.
     * Requisitos:
     * - NODE deve ter: static merge(const NODE&, const NODE&),
     *   apply(const TAG&, ll, ll) e construtor identidade.
     * - TAG deve ter: apply(const TAG&), campo 'has' (bool) e
     *   construtor identidade.
     */
    /* --- Exemplo de NODE e TAG ---
    struct Tag {
        ll add = 0;
        bool has = false;
        void apply(const Tag& t) {
            add += t.add;
            has = true;
        }
    };
    struct Node {
        ll val = 0;
        static Node merge(const Node& l, const Node& r) {
            return {l.val + r.val};
        }
        void apply(const Tag& t, ll l, ll r) {
            val += t.add * (r - l + 1);
        }
    };
    */
708 template <typename NODE, typename TAG>
77D struct DynamicSegTree {
126   struct InternalNode {
4EB     NODE node;
657     TAG tag;
74F     int l, r;
FB6     InternalNode() : node(NODE()), tag(TAG()), l(-1), r(-1) {}
A61     void apply(const TAG &t, ll l, ll r) {
BE6       node.apply(t, l, r);
D21       tag.apply(t);
9C3     }
2FB   };
9B3   ll L, R;
4C1   vector<InternalNode> seg;
5C7   DynamicSegTree(ll l, ll r) : L(l), R(r) {
7A0     seg.reserve(4e6);
23E     seg.emplace_back();
BFD   }
4F9   void push(int v, ll l, ll r) {
A16     if (!seg[v].tag.has) return;
56B     if (l == r) return seg[v].tag = TAG();
4D5     if (seg[v].l == -1) {
9EF       seg[v].l = seg.size();
23E       seg.emplace_back();
579     }
CF3     if (seg[v].r == -1) {
90C       seg[v].r = seg.size();
23E       seg.emplace_back();
203     }
769     ll mid = l + (r - l) / 2;
F3E     seg[seg[v].l].apply(seg[v].tag, l, mid);
0B7     seg[seg[v].r].apply(seg[v].tag, mid + 1, r);
DD0     seg[v].tag = TAG();
AD7   }
```

```
352   void update(int v, ll l, ll r, ll ql, ll qr, const TAG &t) {
151     if (ql <= l && r <= qr) return seg[v].apply(t, l, r);
A48     push(v, l, r);
769     ll mid = l + (r - l) / 2;
441     if (ql <= mid) {
4D5       if (seg[v].l == -1) {
9EF         seg[v].l = seg.size();
23E         seg.emplace_back();
579       }
2DD       update(seg[v].l, l, mid, ql, qr, t);
0B1     }
64D     if (qr > mid) {
CF3       if (seg[v].r == -1) {
90C         seg[v].r = seg.size();
23E         seg.emplace_back();
203       }
76B       update(seg[v].r, mid + 1, r, ql, qr, t);
216     }
671     NODE lc = (seg[v].l != -1) ? seg[seg[v].l].node : NODE();
8A0     NODE rc = (seg[v].r != -1) ? seg[seg[v].r].node : NODE();
37A     seg[v].node = NODE::merge(lc, rc);
BF0   }
EE0   NODE query(int v, ll l, ll r, ll ql, ll qr) {
5CF     if (v == -1 || ql > r || qr < l) return NODE();
3C9     if (ql <= l && r <= qr) return seg[v].node;
A48     push(v, l, r);
769     ll mid = l + (r - l) / 2;
211     return NODE::merge(query(seg[v].l, l, mid, ql, qr),
0CC                        query(seg[v].r, mid + 1, r, ql, qr));
933   }
FEE   void update(ll ql, ll qr, const TAG &t) {
B79     update(0, L, R, ql, qr, t);
4CA   }
323   NODE query(ll ql, ll qr) { return query(0, L, R, ql, qr); }
DBC };
```

Dynamic Persistent Segment Tree

```
    /* Persistent Dynamic Segment Tree (Point Update, Range Query)
     * Consulta versoes anteriores e aloca nos sob demanda.
     * Ideal para intervalos grandes e multiplas versoes.
     * Complexidade: O(log N) por update e query.
     * Memoria: O(Q log N) onde Q e o numero de updates.
     * Requisitos:
     * - NODE deve ter: static merge(const NODE&, const NODE&) e
     *   construtor identidade.
     */
    /* --- Exemplo de NODE (Soma) ---
    struct Node {
        int val;
        Node(int v = 0) : val(v) {}
        static Node merge(const Node& l, const Node& r) {
            return Node(l.val + r.val);
        }
    };
    */
8A0 template <typename NODE>
68A struct PersistentSegmentTree {
126   struct InternalNode {
1DF     NODE data;
74F     int l, r;
E61     InternalNode() : data(NODE()), l(0), r(0) {}
B12   };
060   int N;
788   vector<InternalNode> st;
34F   vector<int> roots;
BC9   PersistentSegmentTree(int n) : N(n) {
4E7     st.reserve(4e6);
F96     st.emplace_back();
405     roots.push_back(0);
71E   }
D69   int update(int root, int L, int R, int idx, NODE v) {
4C0     int node = st.size();
DF4     if (root == 0) st.emplace_back();
8FC     else st.push_back({st[root]});
651     if (L == R) {
DC5       st[node].data = NODE::merge(st[node].data, v);
192       return node;
BE8     }
08E     int mid = L + (R - L) / 2;
        // A ordem do LHS importa antes do C++17.
D33     if (idx <= mid) {
561       int prev = (root == 0) ? 0 : st[root].l;
339       st[node].l = update(prev, L, mid, idx, v);
413     } else {
AFD       int prev = (root == 0) ? 0 : st[root].r;
674       st[node].r = update(prev, mid + 1, R, idx, v);
066     }
98D     auto& nd = st[node];
0EB     nd.data = NODE::merge(st[nd.l].data, st[nd.r].data);
192     return node;
60F   }
082   NODE query(int node, int L, int R, int i, int j) {
0B8     if (node == 0 || i > R || j < L) return NODE();
692     if (i <= L && R <= j) return st[node].data;
08E     int mid = L + (R - L) / 2;
B1E     return NODE::merge(query(st[node].l, L, mid, i, j),
521                        query(st[node].r, mid + 1, R, i, j));
26E   }
D6B   void update(int idx, NODE v) {
69E     roots.push_back(update(roots.back(), 0, N - 1, idx, v));
1BF   }
3A5   NODE query(int version, int l, int r) {
191     return query(roots[version], 0, N - 1, l, r);
247   }
B56 };
```

Line Container (CHT)

```
    /* Convex Hull Trick dinamico — O(log N) por operacao
     *
     * Mantem retas y = k·x + m e responde o MAXIMO em x.
     * Aceita k e x arbitrarios; nao exige monotonicidade.
     *
     * Uso: DP dp[i] = max_j(k[j]·x[i] + m[j]). Cada estado vira
     * add(k[j], m[j]); a transicao vira query(x[i]).
     *
     *   add(k, m)  → insere a reta y = k·x + m
     *   query(x)   → maior valor entre todas as retas em x
     *
     * Para MÍNIMO: insira add(-k, -m) e use -query(x).
     */
72C struct Line {
F3E   ll k, m;
6F3   mutable ll p;
CA5   bool operator<(const Line &o) const { return k < o.k; }
ABF   bool operator<(ll x) const { return p < x; }
868 };
781 struct LineContainer : multiset<Line, less<>> {
FD2   static const ll inf = LLONG_MAX;
10F   ll div(ll a, ll b) { return a / b - ((a ^ b) < 0 && a % b); }
A1C   bool isect(iterator x, iterator y) {
A95     if (y == end()) return x->p = inf, 0;
9CB     if (x->k == y->k) x->p = x->m > y->m ? inf : -inf;
591     else x->p = div(y->m - x->m, x->k - y->k);
870     return x->p >= y->p;
2FA   }
A0C   void add(ll k, ll m) {
116     auto z = insert({k, m, 0}), y = z++, x = y;
7B1     while (isect(y, z)) z = erase(z);
141     if (x != begin() && isect(--x, y)) isect(x, y = erase(y));
57D     while ((y = x) != begin() && (--x)->p >= y->p)
774       isect(x, erase(y));
086   }
4AD   ll query(ll x) {
229     assert(!empty());
7D1     auto l = *lower_bound(x);
96A     return l.k * x + l.m;
D21   }
577 };
```

Graphs

Dijkstra

```
    /* Dijkstra — Caminho minimo de fonte unica
     * Complexidade: O((V + E) log V)
     * Requer pesos nao-negativos.
     *
     * adj[u] = lista de {v, w}: aresta u→v com peso w
     *
     * dist[v] = distancia minima de s ate v.
     * dist[v] = INF se v e inalcancavel.
     */
67A template <typename T>
5D5 vector<T> dijkstra(const vector<vector<pair<int, T>>> &adj,
E60                    int s) {
B4C   int n = adj.size();
305   const T INF = numeric_limits<T>::max();
835   vector<T> dist(n, INF);
69C   priority_queue<pair<T, int>, vector<pair<T, int>>, greater<>>
018     pq;
A93   dist[s] = 0;
7BA   pq.push({0, s});
502   while (!pq.empty()) {
2F9     auto [d, u] = pq.top();
716     pq.pop();
3E1     if (d > dist[u]) continue;
610     for (auto [v, w] : adj[u])
DDE       if (dist[u] + w < dist[v]) {
491         dist[v] = dist[u] + w;
BF3         pq.push({dist[v], v});
D74       }
9AE   }
8D7   return dist;
6D3 }
```

MST (Kruskal)

```
32D #include "dsu.hpp"
    /* MST — Arvore Geradora Minima (Kruskal + DSU)
     * Complexidade: O(E log E). Grafo nao-direcionado ponderado.
     *
     * mst(n, edges) → {custo total, arestas usadas}
     *   edges: vetor de {peso, u, v}.
     * Se desconexo, devolve a floresta minima; used tem < n-1.
     *
     * Para arvore MAXIMA, negue os pesos ou ordene decrescente.
     */
67A template <typename T>
FD5 pair<T, vector<pair<int, int>>>
652 mst(int n, vector<tuple<T, int, int>> edges) {
FF1   sort(edges.begin(), edges.end());
370   DSU dsu(n);
C80   T total = 0;
24A   vector<pair<int, int>> used;
DC1   for (auto &[w, u, v] : edges)
5DF     if (dsu.join(u, v)) {
532       total += w;
```

```
12C       used.push_back({u, v});
88B     }
0CF   return {total, used};
290 }
```

Euler Path

```
    /* Caminho/Circuito Euleriano — Hierholzer — O(V + E)
     *
     * Condicoes de existencia:
     *   Grafo nao-direcionado:
     *     - Circuito: todos os vertices tem grau par.
     *     - Caminho: exatamente dois tem grau impar.
     *   Grafo direcionado:
     *     - Circuito: in_degree[v] == out_degree[v] para todo v.
     *     - Caminho: um vertice com out-in=1, um com in-out=1.
     *
     * euler(adj, s) → caminho com E+1 vertices; vazio se invalido.
     * adj[u] = lista de {v, edge_id} para arestas u→v.
     * No nao-direcionado, insira ambas as direcoes com o mesmo id.
     */
E48 vector<int> euler(vector<vector<pair<int, int>>> &adj, int s) {
CE9   int edges = 0;
14F   for (auto &v : adj) edges += v.size();
      // Para direcionado, remova a divisao por 2.
2D5   edges /= 2;
B27   vector<bool> used(edges, false);
8E1   vector<int> ptr(adj.size(), 0);
DAB   vector<int> path, stk = {s};
07D   while (!stk.empty()) {
9E9     int v = stk.back();
2C4     bool found = false;
EBE     while (ptr[v] < (int)adj[v].size()) {
FFC       auto [u, eid] = adj[v][ptr[v]++];
AC3       if (!used[eid]) {
C3D         used[eid] = true;
967         stk.push_back(u);
E96         found = true;
C2B         break;
BE7       }
7CB     }
8C1     if (!found) {
80A       path.push_back(v);
518       stk.pop_back();
9D9     }
8D6   }
66D   if ((int)path.size() != edges + 1) return {};
3A9   reverse(path.begin(), path.end());
535   return path;
924 }
```

Lca Sparse Table

```
D7B #include "sparse-table.hpp"
    /* LCA (Lowest Common Ancestor) via RMQ
     * Ancestral comum mais proximo de dois vertices da arvore.
     * Complexidade: O(N log N) no build, O(1) por query.
     * Memoria: O(N log N)
     * Requisitos:
     * - Grafo deve ser uma arvore (conexo e aciclico).
     * Metodos:
     * - lca(u, v): Retorna o LCA de u e v.
     * - dist(u, v): Retorna a distancia entre u e v.
     * - is_ancestor(u, v): Verifica se u e ancestral de v.
     * - parent(u): Retorna o pai de u.
     */
542 struct LCANode {
43D   int dep, id;
89D   LCANode(int d = 1e9, int pos = -1) : dep(d), id(pos) {}
3E4   static LCANode merge(const LCANode &l, const LCANode &r) {
DC4     return l.dep < r.dep ? l : r;
0A6   }
008 };
33E struct LCA {
060   int N;
41B   vector<int> first, tour, d, p;
73B   SparseTable<LCANode> st;
DFC   LCA(const vector<vector<int>> &adj, int root = 0)
B9F       : N(adj.size()), first(N), d(N), p(N, -1),
CDD         st(vector<LCANode>()) {
AA1     tour.reserve(2 * N);
52E     vector<LCANode> nodes;
A7F     nodes.reserve(2 * N);
70E     auto dfs = [&](auto self, int u, int par, int dep) -> void {
9A7       p[u] = par;
701       d[u] = dep;
D86       first[u] = tour.size();
FD8       tour.push_back(u);
612       nodes.push_back({dep, (int)tour.size() - 1});
372       for (int v : adj[u]) {
D56         if (v == par) continue;
207         self(self, v, u, dep + 1);
FD8         tour.push_back(u);
612         nodes.push_back({dep, (int)tour.size() - 1});
B41       }
958     };
7D9     dfs(dfs, root, -1, 0);
F2E     st = SparseTable<LCANode>(nodes);
490   }
47F   int parent(int u) { return p[u]; }
310   int lca(int u, int v) {
B63     int l = first[u], r = first[v];
A13     if (l > r) swap(l, r);
369     return tour[st.query(l, r).id];
1E6   }
C11   int dist(int u, int v) {
05C     return d[u] + d[v] - 2 * d[lca(u, v)];
465   }
C22   bool is_ancestor(int u, int v) { return lca(u, v) == u; }
19C };
```

Lca Binary Lifting

```
    /* LCA (Lowest Common Ancestor) via Binary Lifting
     * Ancestral comum mais proximo de dois vertices da arvore.
     * Complexidade: O(N log N) no build, O(log N) por query.
     * Memoria: O(N log N)
     * Requisitos:
     * - Grafo deve ser uma arvore (conexo e aciclico).
     * Metodos:
     * - lca(u, v): Retorna o LCA de u e v.
     * - dist(u, v): Retorna a distancia entre u e v.
     * - is_ancestor(u, v): Verifica se u e ancestral de v.
     * - parent(u): Retorna o pai de u (up[u][0]).
     * - kth_ancestor(u, k): k-esimo ancestral; -1 se nao existe.
     */
33E struct LCA {
8B7   int N, LOG;
5CE   vector<int> d;
5FE   vector<vector<int>> up;
DFC   LCA(const vector<vector<int>> &adj, int root = 0)
839       : N(adj.size()), LOG(__lg(N) + 1), d(N),
F1E         up(N, vector<int>(LOG + 1, -1)) {
91B     auto dfs = [&](auto self, int u, int p, int dep) -> void {
701       d[u] = dep;
F1D       up[u][0] = (p == -1) ? u : p;
B16       for (int j = 1; j <= LOG; j++)
4D5         up[u][j] = up[up[u][j - 1]][j - 1];
372       for (int v : adj[u]) {
730         if (v == p) continue;
207         self(self, v, u, dep + 1);
320       }
3B1     };
7D9     dfs(dfs, root, -1, 0);
5A6   }
579   int parent(int u) { return up[u][0] == u ? -1 : up[u][0]; }
6AE   int kth_ancestor(int u, int k) {
472     if (k > d[u]) return -1;
4C8     for (int j = 0; j <= LOG; j++)
518       if ((k >> j) & 1) u = up[u][j];
03F     return u;
51C   }
310   int lca(int u, int v) {
4F1     if (d[u] < d[v]) swap(u, v);
3B4     int diff = d[u] - d[v];
4C8     for (int j = 0; j <= LOG; j++)
122       if ((diff >> j) & 1) u = up[u][j];
60E     if (u == v) return u;
CAE     for (int j = LOG; j >= 0; j--)
76F       if (up[u][j] != up[v][j]) {
0EF         u = up[u][j];
E4F         v = up[v][j];
D97       }
C15     return up[u][0];
B04   }
C11   int dist(int u, int v) {
05C     return d[u] + d[v] - 2 * d[lca(u, v)];
465   }
C22   bool is_ancestor(int u, int v) { return lca(u, v) == u; }
E8E };
```

Tarjan

```
    /* Tarjan SCC (Strongly Connected Components)
     * Complexidade: O(V + E)
     *
     * Campos publicos apos build():
     *   comp[v]  = SCC de v, em ordem topologica reversa
     *   scc[i]   = lista de vertices do i-esimo SCC
     *   dag      = grafo condensado sem arestas duplicadas
     *   n_scc    = numero de SCCs
     *
     * Se ha aresta SCC a → SCC b, entao comp[a] > comp[b].
     * Para ordem topologica direta, percorra n_scc-1 ... 0.
     */
FA3 struct Tarjan {
058   int n, n_scc = 0;
A41   vector<vector<int>> g, scc, dag;
F7A   vector<int> comp;
770   Tarjan(int n) : n(n), g(n), comp(n, -1) {}
224   Tarjan(const vector<vector<int>> &adj)
422       : n(adj.size()), g(adj), comp(adj.size(), -1) {
6F2     build();
35F   }
C2B   void add_edge(int u, int v) { g[u].push_back(v); }
0A8   void build() {
E7C     vector<int> low(n), num(n, -1), stk;
C05     vector<bool> on_stk(n, false);
2DC     int timer = 0;
36D     function<void(int)> dfs = [&](int v) {
006       low[v] = num[v] = timer++;
B00       stk.push_back(v);
48C       on_stk[v] = true;
E32       for (int u : g[v]) {
403         if (num[u] == -1) {
512           dfs(u);
```

```
C80           low[v] = min(low[v], low[u]);
F19         } else if (on_stk[u]) low[v] = min(low[v], num[u]);
38B       }
46E       if (low[v] == num[v]) {
861         scc.push_back({});
667         while (true) {
B01           int u = stk.back();
518           stk.pop_back();
635           on_stk[u] = false;
65B           comp[u] = n_scc;
588           scc.back().push_back(u);
163           if (u == v) break;
87F         }
385         n_scc++;
6CD       }
5A5     };

830     for (int i = 0; i < n; i++)
917       if (num[i] == -1) dfs(i);

9E4     dag.assign(n_scc, {});
19E     for (int u = 0; u < n; u++)
4FB       for (int v : g[u])
D67         if (comp[u] != comp[v]) dag[comp[u]].push_back(comp[v]);

404     for (auto &adj : dag) {
324       sort(adj.begin(), adj.end());
942       adj.erase(unique(adj.begin(), adj.end()), adj.end());
80E     }
BA2   }
6D7 };
```

Articulations

```
    /* Pontos de Articulacao — Tarjan
     * Complexidade: O(V + E)
     * Grafo nao-direcionado.
     *
     * Sua remocao aumenta o numero de componentes conexas.
     *
     * Campos publicos apos build():
     *   is_ap[v] = true se v e ponto de articulacao
     *   get()    = lista dos pontos de articulacao
     */
E5A struct Articulations {
1A8   int n;
789   vector<vector<int>> g;
BC9   vector<bool> is_ap;

620   Articulations(int n) : n(n), g(n), is_ap(n, false) {}

6F5   Articulations(const vector<vector<int>> &adj)
FDA       : n(adj.size()), g(adj), is_ap(adj.size(), false) {
6F2     build();
B9D   }

58B   void add_edge(int u, int v) {
F2B     g[u].push_back(v);
4D6     g[v].push_back(u);
FFF   }

0A8   void build() {
AC3     vector<int> num(n, -1), low(n);
2DC     int timer = 0;

CF3     function<void(int, int)> dfs = [&](int v, int par) {
006       low[v] = num[v] = timer++;
E07       int children = 0;
E32       for (int u : g[v]) {
403         if (num[u] == -1) {
C5F           children++;
412           dfs(u, v);
C80           low[v] = min(low[v], low[u]);
369           if (par == -1 && children > 1) is_ap[v] = true;
1B3           if (par != -1 && low[u] >= num[v]) is_ap[v] = true;
709         } else if (u != par) {
2D3           low[v] = min(low[v], num[u]);
933         }
9F1       }
8CA     };

830     for (int i = 0; i < n; i++)
10B       if (num[i] == -1) dfs(i, -1);
DEE   }

      // Lista dos pontos de articulacao.
D3D   vector<int> get() const {
02F     vector<int> res;
830     for (int i = 0; i < n; i++)
BA3       if (is_ap[i]) res.push_back(i);
B50     return res;
60A   }
A5D };
```

Bridges

```
    /* Pontes (Bridges) — Tarjan
     * Complexidade: O(V + E)
     * Grafo nao-direcionado.
     *
     * Uma aresta e ponte se sua remocao desconecta o grafo.
     *
     * Campos publicos apos build():
     *   bridges = lista de arestas {u, v} que sao pontes
     */
D44 struct Bridges {
1A8   int n;
789   vector<vector<int>> g;
0CC   vector<pair<int, int>> bridges;

B0A   Bridges(int n) : n(n), g(n) {}

C65   Bridges(const vector<vector<int>> &adj)
30E       : n(adj.size()), g(adj) {
6F2     build();
5F7   }

58B   void add_edge(int u, int v) {
F2B     g[u].push_back(v);
4D6     g[v].push_back(u);
FFF   }

0A8   void build() {
AC3     vector<int> num(n, -1), low(n);
2DC     int timer = 0;

CF3     function<void(int, int)> dfs = [&](int v, int par) {
006       low[v] = num[v] = timer++;
E32       for (int u : g[v]) {
403         if (num[u] == -1) {
412           dfs(u, v);
C80           low[v] = min(low[v], low[u]);
2FE           if (low[u] > num[v]) bridges.push_back({v, u});
FB4         } else if (u != par) {
2D3           low[v] = min(low[v], num[u]);
E66         }
7A3       }
938     };

830     for (int i = 0; i < n; i++)
10B       if (num[i] == -1) dfs(i, -1);
A6F   }
000 };
```

2-SAT

```
C31 #include "tarjan.hpp"

    /* 2-SAT — Complexidade: O(V + E)
     *
     * Literal x: use i para x_i=true, ~i para x_i=false.
     *
     *   add_impl(x, y) → x → y e contrapositiva ~y → ~x
     *   add_or(x, y)    → x v y
     *   add_xor(x, y)   → exatamente um de x, y e true
     *   add_iff(x, y)   → x ↔ y
     *   force(x)        → x = true
     *   solve()         → {satisfativel, ans[]}; ans[i] = 0 ou 1
     */
D9D struct TwoSat {
1A8   int n;
0D7   Tarjan t;
DAB   vector<int> ans;

46B   TwoSat(int n) : n(n), t(2 * n), ans(n, -1) {}

      // Converte i em 2*i e ~i em 2*i+1.
5D7   static int node(int x) { return x >= 0 ? 2 * x : -2 * x - 1; }

      // implicacao x → y (e contrapositiva ~y → ~x)
74A   void add_impl(int x, int y) {
D64     int nx = node(x), ny = node(y);
2B2     t.add_edge(nx, ny);
378     t.add_edge(ny ^ 1, nx ^ 1);
255   }

      // x v y
F10   void add_or(int x, int y) { add_impl(~x, y); }

      // x XOR y (exatamente um verdadeiro)
487   void add_xor(int x, int y) {
27A     add_or(x, y);
0E8     add_or(~x, ~y);
C02   }

      // x ↔ y
1FF   void add_iff(int x, int y) {
2D0     add_impl(x, y);
EA9     add_impl(~x, ~y);
721   }

      // forca x = true
5F8   void force(int x) { add_impl(~x, x); }

A8E   pair<bool, vector<int>> solve() {
2BD     t.build();
E1C     ans.assign(n, -1);
603     for (int i = 0; i < n; i++) {
EBA       if (t.comp[2 * i] == t.comp[2 * i + 1]) return {false, {}};
049       ans[i] = t.comp[2 * i] < t.comp[2 * i + 1] ? 1 : 0;
EF0     }
997     return {true, ans};
194   }
AC6 };
```

Hld Commutative

```
0FA #include "segment-tree.hpp"

    /* HLD Comutativo (Update Pontual)
     * Para operacoes onde merge(A, B) = merge(B, A).
     */
8A0 template <typename NODE>
403 struct HLD {
577   int n, t;
523   vector<int> p, sz, d, head, pos;
115   SegTree<NODE> st;

AA9   HLD(const vector<vector<int>> &adj, const vector<NODE> &vals,
44C       int root = 0)
F92       : n(adj.size()), t(0), p(n), sz(n), d(n), head(n), pos(n),
AC1         st(n) {
898     vector<vector<int>> g = adj;
227     d[root] = 0, p[root] = root;

94B     auto dfs_sz = [&](auto self, int u) -> void {
267       sz[u] = 1;
839       int best_v = -1, max_sz = -1;
A8C       for (auto &v : g[u]) {
567         if (v == p[u]) continue;
C2C         d[v] = d[u] + 1;
E42         p[v] = u;
033         self(self, v);
```

```
CC3         sz[u] += sz[v];
1F6         if (sz[v] > max_sz) {
F66           max_sz = sz[v];
DD0           best_v = v;
D9D         }
A7E       }
2D5       if (best_v != -1) {
BB6         for (int i = 0; i < (int)g[u].size(); i++) {
F10           if (g[u][i] == best_v && i != 0) {
D76             swap(g[u][0], g[u][i]);
C2B             break;
27A           }
CAE         }
07D       }
A11     };
B9B     auto dfs_hld = [&](auto self, int u) -> void {
4C8       pos[u] = t++;
BB6       for (int i = 0; i < (int)g[u].size(); i++) {
06E         int v = g[u][i];
567         if (v == p[u]) continue;
BD7         head[v] = (i == 0 ? head[u] : v);
033         self(self, v);
D08       }
113     };
A8A     dfs_sz(dfs_sz, root);
C07     head[root] = root;
C37     dfs_hld(dfs_hld, root);
759     vector<NODE> base(n);
852     for (int i = 0; i < n; i++) base[pos[i]] = vals[i];
E4E     st = SegTree<NODE>(base);
1E0   }
080   void update(int u, const NODE &val) { st.update(pos[u], val); }
0F8   NODE query_path(int u, int v) {
ADA     NODE res;
0E0     while (head[u] != head[v]) {
44B       if (d[head[u]] > d[head[v]]) swap(u, v);
A0D       res = NODE::merge(res, st.query(pos[head[v]], pos[v]));
025       v = p[head[v]];
41F     }
0E0     if (d[u] > d[v]) swap(u, v);
DC7     return NODE::merge(res, st.query(pos[u], pos[v]));
81B   }
51A   NODE query_subtree(int u) {
0CF     return st.query(pos[u], pos[u] + sz[u] - 1);
747   }
058 };
```

Hld Commutative Lazy

```
064 #include "lazy-segment-tree.hpp"
    /* HLD Comutativo com Lazy Propagation
     * Para operacoes onde merge(A, B) = merge(B, A).
     * Updates e queries em caminhos e subarvores.
     */
708 template <typename NODE, typename TAG>
403 struct HLD {
577   int n, t;
523   vector<int> p, sz, d, head, pos;
FFF   LazySegmentTree<NODE, TAG> st;
AA9   HLD(const vector<vector<int>> &adj, const vector<NODE> &vals,
44C       int root = 0)
F92       : n(adj.size()), t(0), p(n), sz(n), d(n), head(n), pos(n),
AC1         st(n) {
898     vector<vector<int>> g = adj;
227     d[root] = 0, p[root] = root;
94B     auto dfs_sz = [&](auto self, int u) -> void {
267       sz[u] = 1;
839       int best_v = -1, max_sz = -1;
A8C       for (auto &v : g[u]) {
567         if (v == p[u]) continue;
C2C         d[v] = d[u] + 1;
E42         p[v] = u;
033         self(self, v);
CC3         sz[u] += sz[v];
1F6         if (sz[v] > max_sz) {
F66           max_sz = sz[v];
DD0           best_v = v;
D9D         }
A7E       }
2D5       if (best_v != -1) {
BB6         for (int i = 0; i < (int)g[u].size(); i++) {
F10           if (g[u][i] == best_v && i != 0) {
D76             swap(g[u][0], g[u][i]);
C2B             break;
27A           }
CAE         }
07D       }
A11     };
B9B     auto dfs_hld = [&](auto self, int u) -> void {
4C8       pos[u] = t++;
BB6       for (int i = 0; i < (int)g[u].size(); i++) {
06E         int v = g[u][i];
567         if (v == p[u]) continue;
BD7         head[v] = (i == 0 ? head[u] : v);
033         self(self, v);
D08       }
113     };
A8A     dfs_sz(dfs_sz, root);
C07     head[root] = root;
C37     dfs_hld(dfs_hld, root);
759     vector<NODE> base(n);
852     for (int i = 0; i < n; i++) base[pos[i]] = vals[i];
9F8     st = LazySegmentTree<NODE, TAG>(base);
631   }
D7C   void update_path(int u, int v, const TAG &tag) {
0E0     while (head[u] != head[v]) {
44B       if (d[head[u]] > d[head[v]]) swap(u, v);
EC8       st.update(pos[head[v]], pos[v], tag);
025       v = p[head[v]];
DD0     }
0E0     if (d[u] > d[v]) swap(u, v);
07A     st.update(pos[u], pos[v], tag);
656   }
0F8   NODE query_path(int u, int v) {
ADA     NODE res;
0E0     while (head[u] != head[v]) {
44B       if (d[head[u]] > d[head[v]]) swap(u, v);
A0D       res = NODE::merge(res, st.query(pos[head[v]], pos[v]));
025       v = p[head[v]];
41F     }
0E0     if (d[u] > d[v]) swap(u, v);
DC7     return NODE::merge(res, st.query(pos[u], pos[v]));
81B   }
F81   void update_subtree(int u, const TAG &tag) {
675     st.update(pos[u], pos[u] + sz[u] - 1, tag);
20D   }
51A   NODE query_subtree(int u) {
0CF     return st.query(pos[u], pos[u] + sz[u] - 1);
747   }
AA5 };
```

Hld Non Commutative

```
0FA #include "segment-tree.hpp"
    /* HLD Nao-Comutativo (Update Pontual)
     * Para merge(A, B) != merge(B, A).
     * DoubleNode guarda as duas direcoes do merge.
     */
8A0 template <typename NODE>
0FF struct DoubleNode {
A30   NODE down, up;
F17   DoubleNode() : down(NODE()), up(NODE()) {}
77F   DoubleNode(const NODE &n) : down(n), up(n) {}
31A   DoubleNode(const NODE &d, const NODE &u) : down(d), up(u) {}
25A   static DoubleNode merge(const DoubleNode &l,
553                           const DoubleNode &r) {
7E0     return {NODE::merge(l.down, r.down),
5D3             NODE::merge(r.up, l.up)};
063   }
FB7 };
8A0 template <typename NODE>
403 struct HLD {
577   int n, t;
256   vector<int> p, sz, d, head, pos, tour;
58B   SegTree<DoubleNode<NODE>> st;
AA9   HLD(const vector<vector<int>> &adj, const vector<NODE> &vals,
44C       int root = 0)
F92       : n(adj.size()), t(0), p(n), sz(n), d(n), head(n), pos(n),
007         tour(n), st(n) {
898     vector<vector<int>> g = adj;
227     d[root] = 0, p[root] = root;
94B     auto dfs_sz = [&](auto self, int u) -> void {
267       sz[u] = 1;
839       int best_v = -1, max_sz = -1;
A8C       for (auto &v : g[u]) {
567         if (v == p[u]) continue;
C2C         d[v] = d[u] + 1;
E42         p[v] = u;
033         self(self, v);
CC3         sz[u] += sz[v];
1F6         if (sz[v] > max_sz) {
F66           max_sz = sz[v];
DD0           best_v = v;
D9D         }
A7E       }
2D5       if (best_v != -1) {
BB6         for (int i = 0; i < (int)g[u].size(); i++) {
F10           if (g[u][i] == best_v && i != 0) {
D76             swap(g[u][0], g[u][i]);
C2B             break;
27A           }
CAE         }
07D       }
A11     };
B9B     auto dfs_hld = [&](auto self, int u) -> void {
4C8       pos[u] = t++;
6EA       tour[pos[u]] = u;
BB6       for (int i = 0; i < (int)g[u].size(); i++) {
06E         int v = g[u][i];
567         if (v == p[u]) continue;
BD7         head[v] = (i == 0 ? head[u] : v);
033         self(self, v);
D08       }
54A     };
A8A     dfs_sz(dfs_sz, root);
C07     head[root] = root;
C37     dfs_hld(dfs_hld, root);
9FC     vector<DoubleNode<NODE>> base(n);
830     for (int i = 0; i < n; i++)
9A8       base[pos[i]] = DoubleNode<NODE>(vals[i]);
19C     st = SegTree<DoubleNode<NODE>>(base);
05A   }
E1B   void update(int u, const NODE &val) {
353     st.update(pos[u], DoubleNode<NODE>(val));
443   }
```

```
0F8   NODE query_path(int u, int v) {
716     NODE l, r;
0E0     while (head[u] != head[v]) {
BF1       if (d[head[u]] > d[head[v]]) {
0D0         l = NODE::merge(l, st.query(pos[head[u]], pos[u]).up);
139         u = p[head[u]];
A08       } else {
455         r = NODE::merge(st.query(pos[head[v]], pos[v]).down, r);
025         v = p[head[v]];
A09       }
02B     }
152     if (d[u] > d[v]) {
AF0       l = NODE::merge(l, st.query(pos[v], pos[u]).up);
EF6     } else {
5AF       r = NODE::merge(st.query(pos[u], pos[v]).down, r);
F3A     }
79F     return NODE::merge(l, r);
22A   }
51A   NODE query_subtree(int u) {
DE6     return st.query(pos[u], pos[u] + sz[u] - 1).down;
588   }
45D };
```

Hld Non Commutative Lazy

```
064 #include "lazy-segment-tree.hpp"

    /* HLD Nao-Comutativo com Lazy Propagation
     * Para merge(A, B) != merge(B, A), como matrizes.
     * DoubleNode guarda as ordens normal e invertida do merge.
     */
8A0 template <typename NODE>
0FF struct DoubleNode {
A30   NODE down, up;
F17   DoubleNode() : down(NODE()), up(NODE()) {}
77F   DoubleNode(const NODE &n) : down(n), up(n) {}
31A   DoubleNode(const NODE &d, const NODE &u) : down(d), up(u) {}
25A   static DoubleNode merge(const DoubleNode &l,
553                           const DoubleNode &r) {
7E0     return {NODE::merge(l.down, r.down),
5D3             NODE::merge(r.up, l.up)};
063   }
BBF   template <typename TAG>
3CC   void apply(const TAG &t, int l, int r) {
B08     down.apply(t, l, r);
2BF     up.apply(t, l, r);
8FB   }
204 };
708 template <typename NODE, typename TAG>
403 struct HLD {
577   int n, t;
523   vector<int> p, sz, d, head, pos;
C94   LazySegmentTree<DoubleNode<NODE>, TAG> st;
AA9   HLD(const vector<vector<int>> &adj, const vector<NODE> &vals,
44C       int root = 0)
F92       : n(adj.size()), t(0), p(n), sz(n), d(n), head(n), pos(n),
AC1         st(n) {
898     vector<vector<int>> g = adj;
227     d[root] = 0, p[root] = root;
94B     auto dfs_sz = [&](auto self, int u) -> void {
267       sz[u] = 1;
839       int best_v = -1, max_sz = -1;
A8C       for (auto &v : g[u]) {
567         if (v == p[u]) continue;
C2C         d[v] = d[u] + 1;
E42         p[v] = u;
033         self(self, v);
CC3         sz[u] += sz[v];
1F6         if (sz[v] > max_sz) {
F66           max_sz = sz[v];
DD0           best_v = v;
D9D         }
A7E       }
2D5       if (best_v != -1) {
BB6         for (int i = 0; i < (int)g[u].size(); i++) {
F10           if (g[u][i] == best_v && i != 0) {
D76             swap(g[u][0], g[u][i]);
C2B             break;
27A           }
CAE         }
07D       }
A11     };
B9B     auto dfs_hld = [&](auto self, int u) -> void {
4C8       pos[u] = t++;
BB6       for (int i = 0; i < (int)g[u].size(); i++) {
06E         int v = g[u][i];
567         if (v == p[u]) continue;
BD7         head[v] = (i == 0 ? head[u] : v);
033         self(self, v);
D08       }
113     };
A8A     dfs_sz(dfs_sz, root);
C07     head[root] = root;
C37     dfs_hld(dfs_hld, root);
9FC     vector<DoubleNode<NODE>> base(n);
830     for (int i = 0; i < n; i++)
9A8       base[pos[i]] = DoubleNode<NODE>(vals[i]);
54E     st = LazySegmentTree<DoubleNode<NODE>, TAG>(base);
716   }
D7C   void update_path(int u, int v, const TAG &tag) {
0E0     while (head[u] != head[v]) {
BF1       if (d[head[u]] > d[head[v]]) {
D76         st.update(pos[head[u]], pos[u], tag);
139         u = p[head[u]];
D79       } else {
EC8         st.update(pos[head[v]], pos[v], tag);
025         v = p[head[v]];
E8B       }
15B     }
B83     if (d[u] > d[v]) st.update(pos[v], pos[u], tag);
052     else st.update(pos[u], pos[v], tag);
855   }
0F8   NODE query_path(int u, int v) {
716     NODE l, r;
0E0     while (head[u] != head[v]) {
BF1       if (d[head[u]] > d[head[v]]) {
0D0         l = NODE::merge(l, st.query(pos[head[u]], pos[u]).up);
139         u = p[head[u]];
A08       } else {
455         r = NODE::merge(st.query(pos[head[v]], pos[v]).down, r);
025         v = p[head[v]];
A09       }
02B     }
152     if (d[u] > d[v]) {
AF0       l = NODE::merge(l, st.query(pos[v], pos[u]).up);
EF6     } else {
5AF       r = NODE::merge(st.query(pos[u], pos[v]).down, r);
F3A     }
79F     return NODE::merge(l, r);
22A   }
F81   void update_subtree(int u, const TAG &tag) {
675     st.update(pos[u], pos[u] + sz[u] - 1, tag);
20D   }
51A   NODE query_subtree(int u) {
DE6     return st.query(pos[u], pos[u] + sz[u] - 1).down;
588   }
3DA };
```

Dinic

```
    /* Dinic (Max Flow)
     * Complexidade: O(V²E) geral; O(E√V) em grafos unitarios.
     * Memoria: O(V + E)
     *
     * Metodos:
     * add_edge(u, v, cap) → aresta u→v com capacidade cap
     * add_edge(u, v, cap, rcap) → define capacidade reversa
     *   flow(s, t)                → fluxo maximo de s a t
     * min_cut(s) → lado alcancavel da fonte apos flow()
     */
67A template <typename T>
14D struct Dinic {
E9B   struct Edge {
C63     int to, rev;
4AE     T cap;
001   };
060   int N;
ED3   vector<vector<Edge>> g;
538   vector<int> level, iter;
241   Dinic(int n) : N(n), g(n), level(n), iter(n) {}
D8F   void add_edge(int u, int v, T cap, T rcap = 0) {
25C     g[u].push_back({v, (int)g[v].size(), cap});
ACA     g[v].push_back({u, (int)g[u].size() - 1, rcap});
05B   }
123   bool bfs(int s, int t) {
224     fill(level.begin(), level.end(), -1);
26A     queue<int> q;
EC8     level[s] = 0;
08B     q.push(s);
14D     while (!q.empty()) {
B1E       int v = q.front();
833       q.pop();
24E       for (auto &e : g[v])
466         if (e.cap > 0 && level[e.to] < 0) {
BD7           level[e.to] = level[v] + 1;
6F4           q.push(e.to);
A33         }
FF9     }
FCB     return level[t] >= 0;
56A   }
D20   T dfs(int v, int t, T f) {
704     if (v == t) return f;
6E7     for (int &i = iter[v]; i < (int)g[v].size(); i++) {
63C       Edge &e = g[v][i];
2C2       if (e.cap > 0 && level[v] < level[e.to]) {
054         T d = dfs(e.to, t, min(f, e.cap));
1A3         if (d > 0) {
8FC           e.cap -= d;
772           g[e.to][e.rev].cap += d;
BE2           return d;
18D         }
57D       }
E13     }
BB3     return 0;
182   }
903   T flow(int s, int t) {
19A     T res = 0;
8CE     while (bfs(s, t)) {
736       fill(iter.begin(), iter.end(), 0);
A96       T d;
AE9       while ((d = dfs(s, t, numeric_limits<T>::max())) > 0)
337         res += d;
91B     }
B50     return res;
CE8   }
8B0   vector<bool> min_cut(int s) {
F71     vector<bool> vis(N, false);
```

```
26A     queue<int> q;
451     vis[s] = true;
08B     q.push(s);
14D     while (!q.empty()) {
B1E       int v = q.front();
833       q.pop();
24E       for (auto &e : g[v])
F04         if (e.cap > 0 && !vis[e.to]) {
1E8           vis[e.to] = true;
6F4           q.push(e.to);
A56         }
607     }
C81     return vis;
71F   }
993 };
```

Min Cost Max Flow

```
    /* Min-Cost Max-Flow (SPFA/Bellman-Ford based)
     * Complexidade: O(VE·flow) no pior caso.
     * Memoria: O(V + E)
     *
     * Metodos:
     * add_edge(u, v, cap, cost) → aresta u→v com cap e custo
     * add_edge(u, v, cap, rcap, cost) → capacidade reversa rcap
     * flow(s, t) → {fluxo maximo, custo minimo} de s a t
     * flow(s, t, limit) → limita o fluxo total a limit
     *
     * Observacoes:
     * - Suporta custos negativos (usa SPFA em vez de Dijkstra).
     * - Sem custos negativos, Dijkstra+potenciais da O(E log V)/aug.
     * - Retorna o menor custo para o fluxo maximo obtido.
     */
39C template <typename Cap, typename Cost>
6F3 struct MCMF {
E9B   struct Edge {
C63     int to, rev;
2E2     Cap cap;
CB9     Cost cost;
BFF   };
060   int N;
ED3   vector<vector<Edge>> g;
7E6   MCMF(int n) : N(n), g(n) {}
251   void add_edge(int u, int v, Cap cap, Cost cost, Cap rcap = 0) {
C5D     g[u].push_back({v, (int)g[v].size(), cap, cost});
D90     g[v].push_back({u, (int)g[u].size() - 1, rcap, -cost});
200   }
C97   pair<Cap, Cost> flow(int s, int t,
ABA                        Cap limit = numeric_limits<Cap>::max()) {
8AC     Cap total_flow = 0;
8F8     Cost total_cost = 0;
DD0     while (limit > 0) {
715       vector<Cost> dist(N, numeric_limits<Cost>::max());
7A2       vector<bool> in_queue(N, false);
FF9       vector<int> prev_node(N, -1), prev_edge(N, -1);
A93       dist[s] = 0;
26A       queue<int> q;
08B       q.push(s);
6FD       in_queue[s] = true;
14D       while (!q.empty()) {
B1E         int v = q.front();
833         q.pop();
F19         in_queue[v] = false;
775         for (int i = 0; i < (int)g[v].size(); i++) {
027           auto &e = g[v][i];
AB4           if (e.cap > 0 && dist[v] + e.cost < dist[e.to]) {
9B1             dist[e.to] = dist[v] + e.cost;
3C0             prev_node[e.to] = v;
2E2             prev_edge[e.to] = i;
5CA             if (!in_queue[e.to]) {
304               in_queue[e.to] = true;
6F4               q.push(e.to);
8A6             }
5E7           }
168         }
A8B       }
472       if (dist[t] == numeric_limits<Cost>::max()) break;
B96       Cap pushed = limit;
8CE       for (int v = t; v != s; v = prev_node[v])
31C         pushed = min(pushed, g[prev_node[v]][prev_edge[v]].cap);
879       for (int v = t; v != s; v = prev_node[v]) {
49B         auto &e = g[prev_node[v]][prev_edge[v]];
A04         e.cap -= pushed;
B9A         g[v][e.rev].cap += pushed;
F38       }
3BD       total_flow += pushed;
6CE       total_cost += pushed * dist[t];
3FE       limit -= pushed;
B73     }
570     return {total_flow, total_cost};
2AA   }
6F3 };
```

Bipartite Matching

```
0AA #include "dinic.hpp"
    /* Bipartite Matching via Dinic
     * Matching maximo em grafo bipartido.
     * Complexidade: O(E * sqrt(V))
     *
     * Vertices esquerda: [0, n), direita: [n, n+m).
     *
     * Campos publicos apos match():
     *   size   = tamanho do matching maximo
     *   match_l[i] = par de i na direita; -1 se livre
     *   match_r[j] = par de j na esquerda; -1 se livre
     *
     * Metodos:
     *   add_edge(i, j) → aresta entre esquerda i e direita j
     *   match()        → executa o matching, retorna tamanho
     */
878 struct BipartiteMatching {
14E   int n, m;
690   Dinic<int> g;
3AE   int S, T;
2AF   vector<int> match_l, match_r;
689   int size = 0;
1FD   BipartiteMatching(int n, int m)
DD3       : n(n), m(m), g(n + m + 2), S(n + m), T(n + m + 1),
779         match_l(n, -1), match_r(m, -1) {
1D3     for (int i = 0; i < n; i++) g.add_edge(S, i, 1);
CC1     for (int j = 0; j < m; j++) g.add_edge(n + j, T, 1);
B60   }
EE4   void add_edge(int i, int j) { g.add_edge(i, n + j, 1); }
274   int match() {
5D7     size = g.flow(S, T);
830     for (int i = 0; i < n; i++)
B83       for (auto &e : g.g[i])
772         if (e.to != S && e.cap == 0) {
846           match_l[i] = e.to - n;
78E           match_r[e.to - n] = i;
FE7         }
1C6     return size;
B47   }
CFA };
```

Math

Mint

```
    /* mint — Inteiro modular
     * Complexidade: O(1) por operacao, O(log mod) para inv()
     *
     * template<typename T, T mod>
     *
     * Uso comum:
     *   using mint = Mint<int, 998244353>;
     *   using mll  = Mint<ll, 998244353LL>;
     *
     * Operacoes: +, -, *, /, ==, !=, <<
     * inv() → inverso modular (mod deve ser primo)
     */
2DB template <typename T, T mod>
4D0 struct Mint {
CA0   T v;
926   Mint(T v = 0) : v(((v % mod) + mod) % mod) {}
EBE   Mint operator+(Mint o) const {
07D     return v + o.v >= mod ? v + o.v - mod : v + o.v;
E93   }
774   Mint operator-(Mint o) const {
D65     return v - o.v < 0 ? v - o.v + mod : v - o.v;
66C   }
3B1   Mint operator*(Mint o) const { return (ll)v * o.v % mod; }
5B6   Mint operator/(Mint o) const { return *this * o.inv(); }
96C   Mint &operator+=(Mint o) { return *this = *this + o; }
BB0   Mint &operator-=(Mint o) { return *this = *this - o; }
926   Mint &operator*=(Mint o) { return *this = *this * o; }
E1C   Mint &operator/=(Mint o) { return *this = *this / o; }
8D1   bool operator==(Mint o) const { return v == o.v; }
587   bool operator!=(Mint o) const { return v != o.v; }
A45   explicit operator T() const { return v; }
EC5   Mint inv() const {
FD5     T a = v, b = mod, x = 1, y = 0;
1B4     while (b) {
295       T q = a / b;
617       a -= q * b;
257       swap(a, b);
67E       x -= q * y;
9DD       swap(x, y);
367     }
EA5     return x;
0CF   }
617   friend ostream &operator<<(ostream &os, Mint m) {
B1C     return os << m.v;
09A   }
AAA };
```

Fast Exponentiation

```
    /* fexp — Exponenciacao rapida modular
     * Complexidade: O(log b)
     *
     * fexp(a, b, mod) → a^b % mod
     * modinv(a, mod) → a^{-1} % mod; requer mod primo (Fermat)
     *
     * Usa __int128 no produto: seguro para mod ate ~9.2·10¹⁸.
     */
42B ll fexp(ll a, ll b, ll mod) {
67C   a %= mod;
3F8   if (a < 0) a += mod;
CE0   ll res = 1;
C71   for (; b > 0; b >>= 1, a = (__int128)a * a % mod)
D47     if (b & 1) res = (__int128)res * a % mod;
B50   return res;
A29 }
```

```
9E3 ll modinv(ll a, ll mod) {
E74   return fexp(a, mod - 2, mod);
64B }
```

Sieve

```
    /* Crivo de Eratostenes
     * Complexidade: O(N log log N)
     *
     * sieve(n) → vetor com todos os primos ate n (inclusive).
     */
705 vector<int> sieve(int n) {
EFA   vector<bool> is_prime(n + 1, true);
FD3   vector<int> primes;
19E   is_prime[0] = is_prime[1] = false;
8A9   for (int p = 2; p <= n; p++) {
29C     if (is_prime[p]) {
BCE       primes.push_back(p);
FAB       if ((ll)p * p <= n)
87F         for (int i = p * p; i <= n; i += p) is_prime[i] = false;
089     }
284   }
A20   return primes;
912 }
    /* Crivo de menor fator primo (SPF) — O(N log log N) build
     * spf[i] = menor primo de i; spf[i] == i indica primo.
     * Permite fatorar qualquer x <= n em O(log x).
     *
     * auto spf = spf_sieve(n);
     * factor(x, spf) → pares {primo, expoente} ordenados.
     */
830 vector<int> spf_sieve(int n) {
78F   vector<int> spf(n + 1);
2B8   iota(spf.begin(), spf.end(), 0);
225   for (int i = 2; (ll)i * i <= n; i++)
DD0     if (spf[i] == i)
C0A       for (int j = i * i; j <= n; j += i)
480         if (spf[j] == j) spf[j] = i;
D4F   return spf;
2DC }
4FF vector<pair<int, int>> factor(int x, const vector<int> &spf) {
490   vector<pair<int, int>> f;
40E   while (x > 1) {
239     int p = spf[x], e = 0;
FA7     while (x % p == 0) {
43F       x /= p;
1A3       e++;
E87     }
6BF     f.push_back({p, e});
760   }
ABE   return f;
B2C }
    /* Divisores de x fora de ordem — O(d(x)).
     * Requer x dentro do crivo usado para gerar spf.
     * Aplique sort() ao retorno quando precisar de ordem.
     */
17D vector<int> divisors(int x, const vector<int> &spf) {
FC1   vector<int> divs = {1};
D07   for (auto [p, e] : factor(x, spf)) {
F49     int sz = divs.size(), pk = 1;
6E5     for (int k = 0; k < e; k++) {
DC5       pk *= p;
BA1       for (int i = 0; i < sz; i++) divs.push_back(divs[i] * pk);
FB5     }
261   }
81C   return divs;
C23 }
```

Euler Phi

```
    /* Crivo Linear — O(N)
     *
     * linear_sieve(n) → {primes, phi}
     *   primes : todos os primos ate n (inclusive)
     *   phi    : phi[i] = φ(i) para i em [0, n]
     */
9A7 pair<vector<int>, vector<int>> linear_sieve(int n) {
303   vector<int> phi(n + 1, 0), primes;
614   vector<bool> composite(n + 1, false);
678   phi[1] = 1;
508   for (int i = 2; i <= n; i++) {
C54     if (!composite[i]) {
E74       primes.push_back(i);
ABD       phi[i] = i - 1;
6DB     }
3B9     for (int p : primes) {
A22       if ((ll)i * p > n) break;
847       composite[i * p] = true;
569       if (i % p == 0) {
8EC         phi[i * p] = phi[i] * p;
C2B         break;
792       } else {
812         phi[i * p] = phi[i] * (p - 1);
988       }
C4A     }
839   }
6EF   return {primes, phi};
415 }
```

Number Theory

```
    /* GCD / LCM — O(log min(a, b)) */
9C4 ll gcd(ll a, ll b) {
D0B   return b ? gcd(b, a % b) : a;
225 }
026 ll lcm(ll a, ll b) {
8B8   return a / gcd(a, b) * b;
15C }
    /* Divisores de n — O(sqrt(N))
     * Retorna lista de divisores em ordem crescente.
     */
5FA vector<ll> divisors(ll n) {
17E   vector<ll> lo, hi;
148   for (ll i = 1; i * i <= n; i++) {
C06     if (n % i == 0) {
1EB       lo.push_back(i);
9C3       if (i != n / i) hi.push_back(n / i);
B5B     }
2DE   }
FC1   reverse(hi.begin(), hi.end());
3AF   lo.insert(lo.end(), hi.begin(), hi.end());
253   return lo;
B9C }
    /* Fatoracao em primos — O(sqrt(N))
     * Retorna lista de {primo, expoente} em ordem crescente.
     */
7E3 vector<pair<ll, int>> factorize(ll n) {
618   vector<pair<ll, int>> factors;
D2E   for (ll p = 2; p * p <= n; p++) {
80A     if (n % p == 0) {
A01       int exp = 0;
03E       while (n % p == 0) {
F4A         n /= p;
666         exp++;
6C9       }
13A       factors.push_back({p, exp});
2E9     }
47D   }
992   if (n > 1) factors.push_back({n, 1});
FA9   return factors;
92A }
    /* CRT — Teorema Chines dos Restos — O(log min(m1, m2))
     * Retorna {x, lcm} com x ≡ a1 (mod m1), x ≡ a2 (mod m2).
     * Retorna {0, 0} se o sistema for incompativel.
     */
8A9 pair<ll, ll> crt(ll a1, ll m1, ll a2, ll m2) {
8F7   ll g = gcd(m1, m2);
916   if ((a2 - a1) % g != 0) return {0, 0};
931   ll mod = m1 / g * m2;
D0B   ll a = m1 / g, b = m2 / g, x = 1, y = 0;
E58   for (ll ta = a, tb = b; tb;) {
B40     ll q = ta / tb;
4AA     ta -= q * tb;
153     swap(ta, tb);
67E     x -= q * y;
9DD     swap(x, y);
54E   }
581   x = (x % (m2 / g) + m2 / g) % (m2 / g);
A62   ll step = (a2 - a1) / g % (m2 / g) * x % (m2 / g);
5FD   ll result = (a1 + m1 * step) % mod;
1EB   return {(result + mod) % mod, mod};
529 }
    /* Fatoracao com primos pre-computados — O(π(sqrt(N)))
     * Util apos sieve(sqrt(n)) para fatorar varios numeros.
     * primes deve conter todos os primos ate sqrt(n).
     */
165 vector<pair<ll, int>> factorize(ll n, const vector<int> &pr) {
618   vector<pair<ll, int>> factors;
38E   for (int p : pr) {
851     if ((ll)p * p > n) break;
80A     if (n % p == 0) {
A01       int exp = 0;
03E       while (n % p == 0) {
F4A         n /= p;
666         exp++;
6C9       }
13A       factors.push_back({p, exp});
2E9     }
6EA   }
992   if (n > 1) factors.push_back({n, 1});
FA9   return factors;
927 }
```

Miller Rabin

```
    /* Miller-Rabin + Pollard Rho
     *
     * is_prime(n) → O(log² n), deterministico para n < 3.3·10¹⁸
     * factorize(n) → O(n^{1/4} log n) esperado; repete fatores
     *
     * Para fatores distintos, use sort + unique.
     */
DA8 ll mulmod(ll a, ll b, ll m) {
C2A   return (__int128)a * b % m;
7D4 }
0A0 bool is_prime(ll n) {
5A7   if (n < 2) return false;
0A8   if (n == 2 || n == 3 || n == 5 || n == 7) return true;
3CE   if (n % 2 == 0 || n % 3 == 0) return false;
AC7   ll d = n - 1;
898   int r = 0;
5B7   while (d % 2 == 0) d /= 2, r++;
C12   for (ll a : {2, 3, 5, 7, 11, 13, 17, 19, 23, 29, 31, 37}) {
89B     if (a >= n) continue;
8C8     ll x = 1, base = a % n, exp = d;
006     for (; exp > 0; exp >>= 1, base = mulmod(base, base, n))
5E6       if (exp & 1) x = mulmod(x, base, n);
8A3     if (x == 1 || x == n - 1) continue;
30D     bool composite = true;
21B     for (int i = 0; i < r - 1; i++) {
F38       x = mulmod(x, x, n);
```

```
A11       if (x == n - 1) {
827         composite = false;
C2B         break;
789       }
4C7     }
4F2     if (composite) return false;
C63   }
8A6   return true;
462 }
711 ll pollard_rho(ll n) {
55A   if (n % 2 == 0) return 2;
ADB   ll x = rand() % (n - 2) + 2, y = x, d = 1;
A4B   ll c = rand() % (n - 1) + 1;
4EB   while (d == 1) {
C17     x = (mulmod(x, x, n) + c) % n;
4EE     y = (mulmod(y, y, n) + c) % n;
4EE     y = (mulmod(y, y, n) + c) % n;
DD8     d = __gcd(abs(x - y), n);
565   }
9EC   return d == n ? pollard_rho(n) : d;
C00 }
6C2 vector<ll> factorize(ll n) {
1B9   if (n == 1) return {};
970   if (is_prime(n)) return {n};
B32   ll d = pollard_rho(n);
8AB   auto l = factorize(d), r = factorize(n / d);
3AF   l.insert(l.end(), r.begin(), r.end());
792   return l;
928 }
```

Matrix

```
    /* Matrix — Multiplicacao e exponenciacao de matriz quadrada
     * Complexidade: O(n³ log k) para M^k
     *
     * Use com mint/mll para operacoes modulares automaticas.
     *
     * Matrix<T> M(n)   → identidade n×n
     * Matrix<T> M(n,0) → matriz n×n zerada
     * M[i][j]          → acesso direto
     * A * B            → multiplicacao O(n³)
     * M ^ k            → M^k por exponenciacao rapida
     */
67A template <typename T>
BF3 struct Matrix {
1A8   int n;
C24   vector<vector<T>> mat;
049   Matrix(int n, int identity = 1)
9AE       : n(n), mat(n, vector<T>(n, T(0))) {
340     if (identity)
3BD       for (int i = 0; i < n; i++) mat[i][i] = T(1);
6ED   }
90D   vector<T> &operator[](int i) { return mat[i]; }
699   const vector<T> &operator[](int i) const { return mat[i]; }
AFB   Matrix operator*(const Matrix &rhs) const {
83B     Matrix res(n, 0);
830     for (int i = 0; i < n; i++)
E22       for (int k = 0; k < n; k++)
E64         if (mat[i][k] != T(0))
F90           for (int j = 0; j < n; j++)
C55             res[i][j] = res[i][j] + mat[i][k] * rhs.mat[k][j];
B50     return res;
893   }
348   Matrix operator^(ll b) const {
2DF     Matrix res(n), a = *this;
0EB     for (; b > 0; b >>= 1, a = a * a)
FC2       if (b & 1) res = res * a;
B50     return res;
1B3   }
ACF };
```

Gaussian Elimination

```
    /* Eliminacao Gaussiana (RREF) — O(N^2 * M)
     * Funciona com double ou Mint<T, mod>.
     *
     * gauss(A, b) → resolve Ax = b.
     * Retorna {x, true} se unica; false nos demais casos.
     */
67A template <typename T>
5D7 pair<vector<T>, bool> gauss(vector<vector<T>> A, vector<T> b) {
FE3   int n = A.size();
3E0   for (int i = 0; i < n; i++) A[i].push_back(b[i]);
3D7   int m = n + 1, rank = 0;
079   for (int col = 0; col < n && rank < n; col++) {
9D3     int sel = rank;
F97     for (int i = rank + 1; i < n; i++)
E75       if (abs(A[i][col]) > abs(A[sel][col])) sel = i;
9E6     if (!(bool)A[sel][col]) continue;
D8D     swap(A[sel], A[rank]);
B46     T iv = T(1) / A[rank][col];
C24     for (int j = col; j < m; j++) A[rank][j] = A[rank][j] * iv;
603     for (int i = 0; i < n; i++) {
BC7       if (i == rank || !(bool)A[i][col]) continue;
BEF       T f = A[i][col];
69A       for (int j = col; j < m; j++)
E47         A[i][j] = A[i][j] - f * A[rank][j];
371     }
56E     rank++;
3F2   }
175   if (rank < n) return {{}, false};
A1B   vector<T> x(n);
B95   for (int i = 0; i < n; i++) x[i] = A[i].back();
079   return {x, true};
724 }
```

Linear Basis

```
    /* Linear Basis (Base Linear XOR)
     * Complexidade: O(B) por operacao; B = numero de bits.
     * Memoria: O(B)
     *
     * Metodos:
     * insert(x)    → true se x e linearmente independente
     * contains(x)  → x pertence ao espaco gerado pela base
     * max_xor(x=0) → maior x XOR subconjunto da base
     * min_xor()    → menor XOR nao-nulo; 0 se base incompleta
     * size() → numero de elementos linearmente independentes
     * merge(other) → insere os elementos de other
     *
     * Observacoes:
     * - Na forma reduzida, cada pivo aparece em uma so linha.
     * - Para o k-esimo XOR, chame reduce() antes de kth().
     */
538 template <typename T = ll, int B = 63>
F45 struct LinearBasis {
E9C   array<T, B + 1> basis{};
1FC   int sz = 0;
220   bool insert(T x) {
C8C     for (int i = B; i >= 0; i--) {
760       if (!((x >> i) & 1)) continue;
617       if (!basis[i]) {
C6C         basis[i] = x;
EB9         sz++;
8A6         return true;
50C       }
6B0       x ^= basis[i];
8A2     }
D1F     return false;
CD7   }
394   bool contains(T x) const {
C8C     for (int i = B; i >= 0; i--) {
760       if (!((x >> i) & 1)) continue;
260       if (!basis[i]) return false;
6B0       x ^= basis[i];
1A9     }
8A6     return true;
189   }
C3B   T max_xor(T x = 0) const {
3CC     for (int i = B; i >= 0; i--) x = max(x, x ^ basis[i]);
EA5     return x;
74B   }
BBB   T min_xor() const {
F56     for (int i = 0; i <= B; i++)
15D       if (basis[i]) return basis[i];
BB3     return 0;
5FC   }
21A   int size() const { return sz; }
AC0   void merge(const LinearBasis &other) {
086     for (int i = B; i >= 0; i--)
D67       if (other.basis[i]) insert(other.basis[i]);
1CA   }
      /* Reduz a base: cada pivo aparece em uma unica linha.
       * Necessario antes de kth(); gera os 2^sz XORs distintos.
       */
93A   void reduce() {
C8C     for (int i = B; i >= 0; i--) {
F14       if (!basis[i]) continue;
E36       for (int j = i + 1; j <= B; j++)
7E7         if ((basis[j] >> i) & 1) basis[j] ^= basis[i];
CA5     }
563   }
      /* k-esimo menor XOR, 0-indexado, entre 2^sz subconjuntos.
       * Requer reduce(). k=0 e 0; k=1 e o menor nao-nulo.
       */
0B4   T kth(ll k) const {
E55     vector<T> elems;
F56     for (int i = 0; i <= B; i++)
210       if (basis[i]) elems.push_back(basis[i]);
        // Ordem crescente do bit mais significativo.
FAC     if (k >= (1LL << (int)elems.size()))
DAA       return -1; // k fora do range
19A     T res = 0;
687     for (int i = 0; i < (int)elems.size(); i++)
A7C       if ((k >> i) & 1) res ^= elems[i];
B50     return res;
B1A   }
DE8 };
```

Unimodal Search

```
    /* Busca unimodal — f deve ser unimodal no intervalo dado.
     *
     * ternary_min/max(lo, hi, f)          → inteiros, O(log N)
     * golden_min/max(lo, hi, f, iters=200) → continuo, O(iters)
     */
    // Ternary search para minimo em inteiros [lo, hi]
398 template <typename F>
7B4 ll ternary_min(ll lo, ll hi, F f) {
960   while (hi - lo > 2) {
27E     ll m1 = lo + (hi - lo) / 3;
D61     ll m2 = hi - (hi - lo) / 3;
2EE     if (f(m1) <= f(m2)) hi = m2;
F67     else lo = m1;
794   }
23F   ll best = lo;
AC1   for (ll x = lo + 1; x <= hi; x++)
EB8     if (f(x) < f(best)) best = x;
F26   return best;
67D }
```

```
    // Ternary search para maximo em inteiros [lo, hi]
398 template <typename F>
810 ll ternary_max(ll lo, ll hi, F f) {
960   while (hi - lo > 2) {
27E     ll m1 = lo + (hi - lo) / 3;
D61     ll m2 = hi - (hi - lo) / 3;
750     if (f(m1) >= f(m2)) hi = m2;
F67     else lo = m1;
F46   }
23F   ll best = lo;
AC1   for (ll x = lo + 1; x <= hi; x++)
592     if (f(x) > f(best)) best = x;
F26   return best;
1C7 }

    // Golden section search para minimo em continuo [lo, hi]
398 template <typename F>
938 double golden_min(double lo, double hi, F f, int iters = 200) {
84E   const double phi = (sqrt(5.0) - 1.0) / 2.0; // ≈ 0.618
0DD   double m1 = hi - phi * (hi - lo);
86C   double m2 = lo + phi * (hi - lo);
87F   double f1 = f(m1), f2 = f(m2);
5B6   for (int i = 0; i < iters; i++) {
F4D     if (f1 < f2) {
82B       hi = m2;
3E8       m2 = m1;
068       f2 = f1;
3D2       m1 = hi - phi * (hi - lo);
8B5       f1 = f(m1);
735     } else {
CEC       lo = m1;
0DE       m1 = m2;
36A       f1 = f2;
C3A       m2 = lo + phi * (hi - lo);
139       f2 = f(m2);
E4A     }
DC3   }
E67   return (lo + hi) / 2.0;
2D2 }

    // Golden section search para maximo em continuo [lo, hi]
398 template <typename F>
CEA double golden_max(double lo, double hi, F f, int iters = 200) {
564   return golden_min(
5F7     lo, hi, [&](double x) { return -f(x); }, iters);
2E7 }
```

FFT

```
316 using CD = complex<ld>;

    /* FFT — Fast Fourier Transform
     * Complexidade: O(N log N)
     *
     * fft(a)       → transforma a no lugar; |a| e potencia de 2
     * conv(a, b)   → convolucao: c[k] = Σ a[i]*b[k-i]
     *
     * Precisao: seguro se (Σa²+Σb²)*log₂N < 9·10¹⁴.
     * Para coeficientes grandes ou modulo, use fft-mod ou NTT.
     */
B4C void fft(vector<CD> &a) {
820   int n = a.size(), log_n = 31 - __builtin_clz(n);

F82   static vector<complex<long double>> R(2, 1);
6B4   static vector<CD> rt(2, 1);
AD8   for (static int k = 2; k < n; k *= 2) {
411     auto x = polar(1.0L, acos(-1.0L) / k);
9D9     R.resize(n);
335     rt.resize(n);
1D3     for (int i = k; i < 2 * k; i++)
CD4       rt[i] = R[i] = i & 1 ? R[i / 2] * x : R[i / 2];
040   }

808   vector<int> rev(n);
830   for (int i = 0; i < n; i++)
F9A     rev[i] = (rev[i / 2] | (i & 1) << log_n) / 2;
830   for (int i = 0; i < n; i++)
A75     if (i < rev[i]) swap(a[i], a[rev[i]]);

657   for (int k = 1; k < n; k *= 2)
1E5     for (int i = 0; i < n; i += 2 * k)
0C2       for (int j = 0; j < k; j++) {
CD2         auto x = (ld *)&rt[j + k], y = (ld *)&a[i + j + k];
259         CD z(x[0] * y[0] - x[1] * y[1],
45D              x[0] * y[1] + x[1] * y[0]);
20A         a[i + j + k] = a[i + j] - z;
1B0         a[i + j] += z;
707       }
960 }

17B vector<ld> conv(const vector<ld> &a, const vector<ld> &b) {
F88   if (a.empty() || b.empty()) return {};
BBB   vector<ld> res(a.size() + b.size() - 1);
E9A   int n = 1 << (32 - __builtin_clz(res.size()));
576   vector<CD> in(n), out(n);
F83   copy(begin(a), end(a), begin(in));
D38   for (int i = 0; i < (int)b.size(); i++) in[i].imag(b[i]);
21A   fft(in);
11C   for (auto &x : in) x *= x;
830   for (int i = 0; i < n; i++)
5DA     out[i] = in[-i & (n - 1)] - conj(in[i]);
3D7   fft(out);
2D6   for (int i = 0; i < (int)res.size(); i++)
A2F     res[i] = imag(out[i]) / (4 * n);
B50   return res;
B6D }
```

FFT Mod

```
240 #include "fft.hpp"

    /* FFT Mod — Convolucao modular com mod arbitrario
     * Complexidade: O(N log N) (~2x mais lento que FFT ou NTT)
     *
     * convMod<mod>(a, b) → c[k] = (Σ a[i]*b[k-i]) % mod
     *
     * Entradas devem estar em [0, mod).
     * Seguro se N·log₂N·mod < 8.6·10¹⁴.
     *
     * Sem mod: use cut = 1<<15 e remova os operadores % mod.
     */
3F9 template <int mod>
2A7 vector<ll> convMod(const vector<ll> &a, const vector<ll> &b) {
F88   if (a.empty() || b.empty()) return {};
290   vector<ll> res(a.size() + b.size() - 1);
07A   int log_n = 32 - __builtin_clz(res.size());
8E1   int n = 1 << log_n;
      // Para alta precisao, use 1<<15 e tire os % mod abaixo.
528   int cut = (int)sqrt(mod);
584   vector<CD> L(n), R(n), outs(n), outl(n);
5B0   for (int i = 0; i < (int)a.size(); i++)
8B7     L[i] = CD((int)a[i] / cut, (int)a[i] % cut);
81B   for (int i = 0; i < (int)b.size(); i++)
F55     R[i] = CD((int)b[i] / cut, (int)b[i] % cut);
5D5   fft(L), fft(R);
603   for (int i = 0; i < n; i++) {
39D     int j = -i & (n - 1);
65E     outl[j] = (L[i] + conj(L[j])) * R[i] / (2.0 * n);
482     outs[j] = (L[i] - conj(L[j])) * R[i] / (2.0 * n) / CD(0, 1);
1C6   }
D08   fft(outl), fft(outs);
6D1   for (int i = 0; i < (int)res.size(); i++) {
        // alta precisao: tire os % mod nas linhas abaixo
54F     ll av = (ll)(real(outl[i]) + .5) % mod;
FA2     ll bv = (ll)(imag(outl[i]) + .5) + (ll)(real(outs[i]) + .5);
A36     ll cv = (ll)(imag(outs[i]) + .5);
557     res[i] = ((av * cut + bv) % mod * cut + cv) % mod;
F04   }
B50   return res;
CA2 }
```

NTT

```
    /* NTT — Number Theoretic Transform
     * Complexidade: O(N log N)
     *
     * NTT<mod, g>::conv(a, b) → c[k] = (Σ a[i]*b[k-i]) % mod
     *
     * Entradas devem estar em [0, mod).
     * mod e primo 2^a·b+1; g e sua raiz primitiva.
     *
     * Opcoes de {mod, g}:
     * {998244353, 3} → 119·2²³+1, mais comum, N ≤ 2²³
     * {469762049, 3} → 7·2²⁶+1, N ≤ 2²⁶
     * {167772161, 3} → 5·2²⁵+1, N ≤ 2²⁵
     * {2013265921, 31} → 15·2²⁷+1, N ≤ 2²⁷ (mod > 10⁹)
     * {2281701377, 3} → 17·2²⁷+1, N ≤ 2²⁷ (mod > 10⁹)
     * {7340033, 5} → 7·2²⁰+1, N ≤ 2²⁰ (menor mod)
     * {9223372036737335297LL,3} → 549755813881·2²⁴+1, N≤2²⁴
     *   (mod~2⁶³, ll)
     */
E34 template <ll mod, ll g>
F12 struct NTT {
D3B   static ll fexp(ll a, ll b) {
CE0     ll res = 1;
67C     a %= mod;
FF0     for (; b > 0; b >>= 1, a = a * a % mod)
602       if (b & 1) res = res * a % mod;
B50     return res;
B0B   }

936   static void ntt(vector<ll> &a) {
820     int n = a.size(), log_n = 31 - __builtin_clz(n);
D51     static vector<ll> rt(2, 1);
8EE     for (static int k = 2, s = 2; k < n; k *= 2, s++) {
335       rt.resize(n);
277       ll z[] = {1, fexp(g, mod >> s)};
1D3       for (int i = k; i < 2 * k; i++)
256         rt[i] = rt[i / 2] * z[i & 1] % mod;
D00     }
808     vector<int> rev(n);
830     for (int i = 0; i < n; i++)
F9A       rev[i] = (rev[i / 2] | (i & 1) << log_n) / 2;
830     for (int i = 0; i < n; i++)
A75       if (i < rev[i]) swap(a[i], a[rev[i]]);
657     for (int k = 1; k < n; k *= 2)
1E5       for (int i = 0; i < n; i += 2 * k)
0C2         for (int j = 0; j < k; j++) {
86E           ll z = rt[j + k] * a[i + j + k] % mod, &ai = a[i + j];
598           a[i + j + k] = ai - z + (z > ai ? mod : 0);
4B8           ai += z - (ai + z >= mod ? mod : 0);
D6A         }
82E   }

46D   static vector<ll> conv(vector<ll> a, vector<ll> b) {
F88     if (a.empty() || b.empty()) return {};
5C2     int s = a.size() + b.size() - 1;
DE2     int n = 1 << (32 - __builtin_clz(s));
667     a.resize(n), b.resize(n); ntt(a), ntt(b);
DDC     ll inv = fexp(n, mod - 2);
6F8     vector<ll> out(n);
830     for (int i = 0; i < n; i++)
50E       out[-i & (n - 1)] = a[i] * b[i] % mod * inv % mod;
EC9     ntt(out);
C20     return {out.begin(), out.begin() + s};
659   }
F65 };
```

FWHT

```
    /* FWHT — Fast Walsh-Hadamard Transform
     * Complexidade: O(N log N)
     *
```

```
 * Convolucao bitwise: c[k] = Σ a[i]*b[j] onde i op j = k
 *
 * fwht_conv<'^'>(a, b) → convolucao XOR
 * fwht_conv<'|'>(a, b) → convolucao OR
 * fwht_conv<'&'>(a, b) → convolucao AND
 *
 * Ajusta os tamanhos para a proxima potencia de 2.
 */
597 template <char op>
823 void fwht(vector<ll> &a, bool inv = false) {
94D   int n = a.size();
980   for (int len = 1; len < n; len *= 2)
EBC     for (int i = 0; i < n; i += 2 * len)
7AB       for (int j = 0; j < len; j++) {
032         ll u = a[i + j], v = a[i + j + len];
FBD         if (op == '^') a[i + j] = u + v, a[i + j + len] = u - v;
02F         if (op == '|') a[i + j + len] = v + (inv ? -u : +u);
729         if (op == '&') a[i + j] = u + (inv ? -v : +v);
1C4       }
609   if (op == '^' && inv)
F33     for (auto &x : a) x /= n;
643 }

597 template <char op>
E0F vector<ll> fwht_conv(vector<ll> a, vector<ll> b) {
43E   int n = 1;
3B9   while (n < (int)max(a.size(), b.size())) n *= 2;
711   a.resize(n);
DD6   b.resize(n);
883   fwht<op>(a);
671   fwht<op>(b);
34E   for (int i = 0; i < n; i++) a[i] *= b[i];
9D7   fwht<op>(a, true);
3F5   return a;
22D }
```

String

KMP

```
/* KMP (Knuth-Morris-Pratt)
 * failure[i] = tamanho do maior prefixo proprio de s[0..i]
 * que e tambem sufixo.
 * Complexidade: O(N) build, O(N + M) busca.
 *
 * Aplicacoes:
 * - Busca de padrao P em texto T em O(|P| + |T|).
 * - Menor periodo da string: N - failure[N-1].
 * - Contar ocorrencias de todos os prefixos como sufixos.
 */
090 vector<int> kmp_failure(const string &s) {
163   int n = s.size();
D11   vector<int> f(n, 0);
6F5   for (int i = 1; i < n; i++) {
844     int j = f[i - 1];
424     while (j > 0 && s[i] != s[j]) j = f[j - 1];
04B     if (s[i] == s[j]) j++;
199     f[i] = j;
D74   }
ABE   return f;
8B7 }

// Índices de todas as ocorrencias de pat em txt.
A0D vector<int> kmp_search(const string &pat, const string &txt) {
921   string s = pat + "$" + txt;
3EC   vector<int> f = kmp_failure(s);
11E   int p = pat.size();
927   vector<int> pos;
D6B   for (int i = p + 1; i < (int)s.size(); i++)
06B     if (f[i] == p) pos.push_back(i - 2 * p);
D75   return pos;
F47 }
```

Z Function

```
/* Z-Function
 * z[i] = maior prefixo de s que coincide com s[i..].
 * z[0] = 0 por convencao.
 * Complexidade: O(N) build.
 *
 * Aplicacoes:
 * - Busca P em T: calcule Z de P + "$" + T.
 * - Ocorrencias de prefixo k: conte z[i] >= k.
 * - Menor periodo: menor p | N com z[p] == N-p.
 */
7AD vector<int> z_function(const string &s) {
163   int n = s.size();
2B1   vector<int> z(n, 0);
A5C   for (int i = 1, l = 0, r = 0; i < n; i++) {
20E     if (i < r) z[i] = min(r - i, z[i - l]);
4ED     while (i + z[i] < n && s[z[i]] == s[i + z[i]]) z[i]++;
B76     if (i + z[i] > r) l = i, r = i + z[i];
3BB   }
070   return z;
197 }

// Índices de todas as ocorrencias de pattern em text.
430 vector<int> z_search(const string &pattern, const string &text) {
365   string s = pattern + "$" + text;
EA6   vector<int> z = z_function(s);
E6D   int p = pattern.size();
FCC   vector<int> positions;
D6B   for (int i = p + 1; i < (int)s.size(); i++)
484     if (z[i] == p) positions.push_back(i - p - 1);
C4C   return positions;
BBF }
```

String Hash

```
/* String Hashing (Single e Double Hash)
 * Hash polinomial: build O(N), comparacao O(1).
 * Complexidade: O(N) build, O(1) query.
 * Memoria: O(N)
 *
 * SingleHash: um par (MOD, BASE), com baixo risco de colisao.
 * DoubleHash: dois pares independentes.
 *
 * Uso:
 *   SingleHash h(s);
 *   h.get(l, r); // hash de s[l..r]
 *   h.get(l, r) == h.get(l2, r2); // compara substrings
 */
/* --- Single Hash --- */
55A struct SingleHash {
029   static const ll MOD = (1LL << 61) - 1;
56C   static const ll BASE;

F42   vector<ll> h, pw;

30E   static ll mul(ll a, ll b) { return (__uint128_t)a * b % MOD; }

67F   static ll add(ll a, ll b) {
9F4     a += b;
BF2     return a >= MOD ? a - MOD : a;
4E8   }

82B   SingleHash() {}

107   SingleHash(const string &s)
2F6       : h(s.size() + 1, 0), pw(s.size() + 1, 1) {
C46     for (int i = 0; i < (int)s.size(); i++) {
4DB       h[i + 1] = add(mul(h[i], BASE), s[i]);
F0F       pw[i + 1] = mul(pw[i], BASE);
0FA     }
485   }

BA8   ll get(int l, int r) const {
128     return add(h[r + 1], MOD - mul(h[l], pw[r - l + 1]));
358   }
4EF };
E2C const ll SingleHash::BASE = []() {
862   mt19937_64 rng(
854     chrono::steady_clock::now().time_since_epoch().count());
2E2   return uniform_int_distribution<ll>(1e9,
86D                                       SingleHash::MOD - 1)(rng);
7E5 }();

/* --- Double Hash --- */
2AF struct DoubleHash {
172   static const ll MOD1 = (1LL << 61) - 1;
D89   static const ll MOD2 = 1e9 + 9;
2BF   static const ll BASE1, BASE2;

23A   vector<ll> h1, h2, pw1, pw2;

780   static ll mul1(ll a, ll b) {
C20     return (__uint128_t)a * b % MOD1;
583   }
876   static ll add1(ll a, ll b) {
9F4     a += b;
B86     return a >= MOD1 ? a - MOD1 : a;
577   }
997   static ll mul2(ll a, ll b) { return (__int128)a * b % MOD2; }
F18   static ll add2(ll a, ll b) {
9F4     a += b;
156     return a >= MOD2 ? a - MOD2 : a;
AC6   }

A3E   DoubleHash() {}

3C9   DoubleHash(const string &s)
01A       : h1(s.size() + 1, 0), h2(s.size() + 1, 0),
4CF         pw1(s.size() + 1, 1), pw2(s.size() + 1, 1) {
C46     for (int i = 0; i < (int)s.size(); i++) {
9F0       h1[i + 1] = add1(mul1(h1[i], BASE1), s[i]);
128       h2[i + 1] = add2(mul2(h2[i], BASE2), s[i]);
562       pw1[i + 1] = mul1(pw1[i], BASE1);
961       pw2[i + 1] = mul2(pw2[i], BASE2);
706     }
366   }

  // Par de hashes de s[l..r], inclusive.
C89   pair<ll, ll> get(int l, int r) const {
DC7     ll a = add1(h1[r + 1], MOD1 - mul1(h1[l], pw1[r - l + 1]));
932     ll b = add2(h2[r + 1], MOD2 - mul2(h2[l], pw2[r - l + 1]));
594     return {a, b};
731   }
3DF };

BBD const ll DoubleHash::BASE1 = []() {
862   mt19937_64 rng(
854     chrono::steady_clock::now().time_since_epoch().count());
2E2   return uniform_int_distribution<ll>(1e9,
263                                       DoubleHash::MOD1 - 1)(rng);
C70 }();

0BD const ll DoubleHash::BASE2 = []() {
862   mt19937_64 rng(
F11     chrono::steady_clock::now().time_since_epoch().count() + 1);
2E2   return uniform_int_distribution<ll>(1e9,
91C                                       DoubleHash::MOD2 - 1)(rng);
13F }();
```

Trie

```
/* Trie (Prefix Tree)
 * Insercao e busca de strings em O(|s|).
 * Memoria: O(N * ALPHA), N = total de caracteres.
 * Alfabeto: [BASE, BASE + ALPHA).
 */
71F template <int ALPHA = 26, char BASE = 'a'>
71A struct Trie {
BF2   struct Node {
C02     int ch[ALPHA];
0F9     int cnt;
302     int end;
```

```
D7F     Node() : cnt(0), end(0) { fill(ch, ch + ALPHA, -1); }
B69   };
F5B   vector<Node> t;
8E7   Trie() { t.emplace_back(); }
E7D   void insert(const string &s) {
3F3     int node = 0;
B4F     for (char c : s) {
E9F       int id = c - BASE;
19E       if (t[node].ch[id] == -1) {
013         t[node].ch[id] = t.size();
83D         t.emplace_back();
838       }
729       node = t[node].ch[id];
59E       t[node].cnt++;
304     }
879     t[node].end++;
2C4   }
BA9   bool search(const string &s) const {
3F3     int node = 0;
B4F     for (char c : s) {
E9F       int id = c - BASE;
D92       if (t[node].ch[id] == -1) return false;
729       node = t[node].ch[id];
047     }
395     return t[node].end > 0;
3FB   }
0B1   int count_prefix(const string &s) const {
3F3     int node = 0;
B4F     for (char c : s) {
E9F       int id = c - BASE;
2AA       if (t[node].ch[id] == -1) return 0;
729       node = t[node].ch[id];
88D     }
D17     return t[node].cnt;
673   }
4E2   int count(const string &s) const {
3F3     int node = 0;
B4F     for (char c : s) {
E9F       int id = c - BASE;
2AA       if (t[node].ch[id] == -1) return 0;
729       node = t[node].ch[id];
88D     }
D51     return t[node].end;
899   }
25C };
```

Manacher

```
    /* Manacher's Algorithm (Palindrome Detection)
     * Encontra todos os palindromos em tempo linear.
     * Complexidade: O(N) onde N e o tamanho da string.
     * Memoria: O(N)
     * Funcoes:
     * - manacher(s): maior palindromo centrado em i.
     * - palindrome: testa s[i..j] em O(1).
     * - pal_begin(s): maior palindromo comecando em i.
     * - pal_end(s): maior palindromo terminando em i.
     */
67A template <typename T>
44D vector<int> manacher(const T &s) {
18F   int l = 0, r = -1, n = s.size();
FC9   vector<int> d1(n), d2(n);
603   for (int i = 0; i < n; i++) {
821     int k = i > r ? 1 : min(d1[l + r - i], r - i);
61A     while (i + k < n && i - k >= 0 && s[i + k] == s[i - k]) k++;
61E     d1[i] = k--;
9F6     if (i + k > r) l = i - k, r = i + k;
950   }
E03   l = 0, r = -1;
603   for (int i = 0; i < n; i++) {
1A6     int k = i > r ? 0 : min(d2[l + r - i + 1], r - i + 1);
AC1     k++;
A94     while (i + k <= n && i - k >= 0 && s[i + k - 1] == s[i - k])
AC1       k++;
EAA     d2[i] = --k;
26D     if (i + k - 1 > r) l = i - k, r = i + k - 1;
4FE   }
1B2   vector<int> res(2 * n - 1);
5CB   for (int i = 0; i < n; i++) res[2 * i] = 2 * d1[i] - 1;
891   for (int i = 0; i < n - 1; i++)
748     res[2 * i + 1] = 2 * d2[i + 1];
B50   return res;
85A }
67A template <typename T>
BF0 struct palindrome {
F97   vector<int> man;
B2D   palindrome(const T &s) : man(manacher(s)) {}
1E7   bool query(int i, int j) { return man[i + j] >= j - i + 1; }
933 };
67A template <typename T>
10D vector<int> pal_begin(const T &s) {
542   int n = s.size() - 1;
A23   vector<int> res(s.size());
FDE   palindrome<T> p(s);
371   res[n] = 1;
45B   for (int i = n - 1; i >= 0; i--) {
D9A     res[i] = min(res[i + 1] + 2, n - i + 1);
F2F     while (!p.query(i, i + res[i] - 1)) res[i]--;
DD9   }
B50   return res;
212 }
67A template <typename T>
934 vector<int> pal_end(const T &s) {
A23   vector<int> res(s.size());
FDE   palindrome<T> p(s);
4EC   res[0] = 1;
88E   for (int i = 1; i < s.size(); i++) {
61D     res[i] = min(res[i - 1] + 2, i + 1);
488     while (!p.query(i - res[i] + 1, i)) res[i]--;
5A2   }
B50   return res;
A2C }
```

Suffix Array (NlogN)

```
    /* Suffix Array O(N log N) — Prefix Doubling com Count Sort
     * sa[i]  = indice do i-esimo sufixo em ordem lexicografica.
     * rk[i]  = rank do sufixo s[i..] (inverso de sa).
     * lcp[i] = LCP entre sa[i-1] e sa[i] (lcp[0] = 0).
     * Adiciona '$' ao final da string internamente.
     * Complexidade: O(N log N) build, O(|P| log N) search.
     *
     * Aplicacoes:
     * - Busca de padrao P: search(p) retorna [lo, hi) no SA.
     * - Substring mais longa repetida: *max_element(lcp).
     * - Numero de substrings distintas: N*(N+1)/2 - sum(lcp).
     * - LCS de duas strings: SuffixArray::lcs(s, t).
     */
3F4 struct SuffixArray {
AC0   string s;
58B   vector<int> sa, rk, lcp;
264   SuffixArray() {}
357   SuffixArray(string s) : s(s) {
6F2     build();
E8F     build_lcp();
A87   }
0A8   void build() {
FC8     s += '$';
163     int n = s.size();
FB9     sa.resize(n);
9BE     rk.resize(n);
413     vector<pair<char, int>> a(n);
490     for (int i = 0; i < n; i++) a[i] = {s[i], i};
3F8     sort(a.begin(), a.end());
67D     for (int i = 0; i < n; i++) sa[i] = a[i].second;
F27     rk[sa[0]] = 0;
6F5     for (int i = 1; i < n; i++) {
7BD       int inc = a[i].first != a[i - 1].first;
6C1       rk[sa[i]] = rk[sa[i - 1]] + inc;
84B     }
8A4     for (int k = 0; (1 << k) < n; k++) {
830       for (int i = 0; i < n; i++)
D82         sa[i] = (sa[i] - (1 << k) + n) % n;
188       count_sort();
037       vector<int> new_rk(n);
CCE       new_rk[sa[0]] = 0;
C6D       auto snd = [&](int x) { return rk[(x + (1 << k)) % n]; };
6F5       for (int i = 1; i < n; i++) {
CD1         pair<int, int> prev = {rk[sa[i - 1]], snd(sa[i - 1])};
3F0         pair<int, int> now = {rk[sa[i]], snd(sa[i])};
AAD         new_rk[sa[i]] = new_rk[sa[i - 1]] + (now != prev);
18C       }
14E       rk = new_rk;
4BE     }
201   }
00C   void count_sort() {
9A8     int n = sa.size();
7C4     vector<int> new_sa(n), cnt(n, 0), pos(n, 0);
CBF     for (auto e : rk) cnt[e]++;
3D7     for (int i = 1; i < n; i++) pos[i] = pos[i - 1] + cnt[i - 1];
1EC     for (auto e : sa) new_sa[pos[rk[e]]++] = e;
253     sa = new_sa;
A28   }
47E   void build_lcp() {
9A8     int n = sa.size();
75C     lcp.assign(n, 0);
617     for (int i = 0, h = 0; i < n; i++) {
F00       if (rk[i] == 0) {
33D         h = 0;
5E2         continue;
595       }
0A2       int j = sa[rk[i] - 1];
28E       while (s[i + h] == s[j + h]) h++;
CA3       lcp[rk[i]] = h;
254       if (h) h--;
7DF     }
2BE   }
      // Intervalo [lo, hi) no SA onde p ocorre.
B04   pair<int, int> search(const string &p) const {
B63     int lo, hi, n = sa.size();
F95     {
993       int l = 0, r = n;
40C       while (l < r) {
EE4         int m = (l + r) / 2;
382         if (s.compare(sa[m], p.size(), p) < 0) l = m + 1;
EF3         else r = m;
103       }
0A8       lo = l;
CBD     }
F95     {
993       int l = 0, r = n;
40C       while (l < r) {
EE4         int m = (l + r) / 2;
E02         if (s.compare(sa[m], p.size(), p) <= 0) l = m + 1;
EF3         else r = m;
D29       }
C1B       hi = l;
277     }
```

```
AAF     return {lo, hi};
EEF   }
      // LCS; sep deve diferir de '$' e nao aparecer em a ou b.
4E7   static string lcs(const string &a, const string &b,
CCC                     char sep = 1) {
B29     SuffixArray suf(a + sep + b);
37E     int m = a.size(), best = 0, pos = 0;
564     for (int i = 1; i < (int)suf.sa.size(); i++) {
935       bool in_a = suf.sa[i] < m;
A7D       bool prev_in_a = suf.sa[i - 1] < m;
B09       if (in_a != prev_in_a && suf.lcp[i] > best) {
F6B         best = suf.lcp[i];
2B8         pos = suf.sa[i];
92E       }
409     }
D5D     return suf.s.substr(pos, best);
514   }
019 };
```

Suffix Array (Linear)

```
    /* Suffix Array O(N) — SA-IS (Induced Sorting)
     * Build O(N). Use quando N muito grande justificar o SA-IS.
     *
     * Campos publicos apos construcao:
     * sa[i] = indice do i-esimo sufixo em ordem lexicografica
     * rk[i] = rank do sufixo s[i..] (inverso de sa)
     * lcp[i] = tamanho do LCP entre sa[i-1] e sa[i]; lcp[0] = 0
     *
     * Metodos:
     * search(p) → [lo, hi) no SA; hi-lo e o nº de ocorrencias
     */
FC8 struct SuffixArrayLinear {
AC0   string s;
58B   vector<int> sa, rk, lcp;
CAE   SuffixArrayLinear() {}
B7A   SuffixArrayLinear(const string &s) : s(s + '$') {
C75     vector<int> t(this->s.begin(), this->s.end());
C9A     sa = sa_is(t, 256);
9A8     int n = sa.size();
9BE     rk.resize(n);
665     for (int i = 0; i < n; i++) rk[sa[i]] = i;
E8F     build_lcp();
B40   }
B04   pair<int, int> search(const string &p) const {
B63     int lo, hi, n = sa.size();
F95     {
993       int l = 0, r = n;
40C       while (l < r) {
EE4         int m = (l + r) / 2;
382         if (s.compare(sa[m], p.size(), p) < 0) l = m + 1;
EF3         else r = m;
103       }
0A8       lo = l;
CBD     }
F95     {
993       int l = 0, r = n;
40C       while (l < r) {
EE4         int m = (l + r) / 2;
E02         if (s.compare(sa[m], p.size(), p) <= 0) l = m + 1;
EF3         else r = m;
D29       }
C1B       hi = l;
277     }
AAF     return {lo, hi};
EEF   }
47E   void build_lcp() {
9A8     int n = sa.size();
75C     lcp.assign(n, 0);
E74     for (int i = 0, h = 0; i < n - 1; i++) {
0A2       int j = sa[rk[i] - 1];
28E       while (s[i + h] == s[j + h]) h++;
CA3       lcp[rk[i]] = h;
254       if (h) h--;
09C     }
C83   }
67A   template <typename T>
EE7   static vector<int> sa_is(const T &s, int sigma) {
163     int n = s.size();
979     if (n == 1) return {0};
4E1     if (n == 2)
32E       return s[0] < s[1] ? vector<int>{0, 1} : vector<int>{1, 0};
81A     vector<bool> t(n);
3A1     t[n - 1] = true;
9C8     for (int i = n - 2; i >= 0; i--)
537       t[i] = s[i] < s[i + 1] || (s[i] == s[i + 1] && t[i + 1]);
A15     auto is_lms = [&](int i) {
BCB       return i > 0 && t[i] && !t[i - 1];
0C4     };
FC6     vector<int> bkt(sigma + 1, 0);
7E5     for (int c : s) bkt[c + 1]++;
CA7     for (int i = 1; i <= sigma; i++) bkt[i] += bkt[i - 1];
CB5     auto induced_sort = [&](const vector<int> &lms) {
386       vector<int> sa(n, -1);
CDA       vector<int> b = bkt;
5EB       for (int i = (int)lms.size() - 1; i >= 0; i--)
F9D         sa[--b[s[lms[i]] + 1]] = lms[i];
332       b = bkt;
830       for (int i = 0; i < n; i++)
DC3         if (sa[i] > 0 && !t[sa[i] - 1])
420           sa[b[s[sa[i] - 1]]++] = sa[i] - 1;
332       b = bkt;
056       for (int i = n - 1; i >= 0; i--)
C0D         if (sa[i] > 0 && t[sa[i] - 1])
3ED           sa[--b[s[sa[i] - 1] + 1]] = sa[i] - 1;
DB7       return sa;
66E     };
021     vector<int> lms;
830     for (int i = 0; i < n; i++)
B39       if (is_lms(i)) lms.push_back(i);
EF0     vector<int> sa = induced_sort(lms);
835     vector<int> rk(n, -1);
8F1     int cls = 0;
975     for (int i = 0, prev = -1; i < n; i++) {
FAD       if (!is_lms(sa[i])) continue;
837       if (prev != -1) {
047         bool diff = false;
5B8         for (int d = 0;; d++) {
12C           if (s[prev + d] != s[sa[i] + d] ||
A89               t[prev + d] != t[sa[i] + d]) {
570             diff = true;
C2B             break;
36D           }
199           if (d > 0 && (is_lms(prev + d) || is_lms(sa[i] + d)))
C2B             break;
DA4         }
265         if (diff) cls++;
142       }
7F0       rk[sa[i]] = cls;
C6D       prev = sa[i];
9FA     }
CDB     vector<int> lms2, rank2;
830     for (int i = 0; i < n; i++)
6A2       if (rk[i] != -1) {
D24         lms2.push_back(i);
213         rank2.push_back(rk[i]);
642       }
        // Ha cls+1 classes distintas, numeradas de 0 a cls.
8F6     vector<int> sa2 = sa_is(rank2, cls + 1);
D14     vector<int> sorted_lms(lms2.size());
E14     for (int i = 0; i < (int)sa2.size(); i++)
42D       sorted_lms[i] = lms2[sa2[i]];
E0A     return induced_sort(sorted_lms);
50A   }
160 };
```

Geometry

Point

```
107 const ld eps = 1e-9;
177 bool eq(ll a, ll b) {
605   return a == b;
6F1 }
D97 bool eq(ld a, ld b) {
BA0   return abs(a - b) <= eps;
BFC }
67A template <typename T>
F26 struct Point {
645   T x, y;
0FA   Point(T x = 0, T y = 0) : x(x), y(y) {}
C93   bool operator<(Point o) const {
AE1     return tie(x, y) < tie(o.x, o.y);
8D2   }
CE2   bool operator==(Point o) const {
651     return eq(x, o.x) && eq(y, o.y);
511   }
9E5   bool operator!=(Point o) const { return !(*this == o); }
480   Point operator+(Point o) const { return {x + o.x, y + o.y}; }
229   Point operator-(Point o) const { return {x - o.x, y - o.y}; }
AE8   Point operator*(T k) const { return {x * k, y * k}; }
7C2   Point operator/(T k) const { return {x / k, y / k}; }
61A   T dot(Point o) const { return x * o.x + y * o.y; }
CB8   T cross(Point o) const { return x * o.y - y * o.x; }
      // area com sinal do triangulo (this, a, b)
DBA   T cross(Point a, Point b) const {
491     return (a - *this).cross(b - *this);
61E   }
F68   T dist2() const { return x * x + y * y; }
AD2   ld len() const { return hypot((ld)x, (ld)y); }
001   Point<ld> norm() const {
DE8     ld l = len();
D97     return {x / l, y / l};
978   }
178   Point perp() const { return {-y, x}; } // rotacao 90° CCW
8CF   Point<ld> rotate(ld a) const {         // a em radianos
97E     return {x * cos(a) - y * sin(a), x * sin(a) + y * cos(a)};
CB4   }
539   friend ostream &operator<<(ostream &os, Point p) {
9B9     return os << p.x << " " << p.y;
DB4   }
2F4 };
29C using Pt = Point<ll>;
BA9 using Ptd = Point<ld>;
    // angulo entre os vetores p e q
67A template <typename T>
AE1 ld angle(Point<T> p, Point<T> q) {
9AF   return atan2((ld)p.cross(q), (ld)p.dot(q));
447 }
    // area com sinal do triangulo (p, q, r) — positivo = CCW
67A template <typename T>
D06 T sarea(Point<T> p, Point<T> q, Point<T> r) {
0CC   return p.cross(q, r);
B80 }
    // se p, q, r sao colineares
```

```
67A template <typename T>
EB4 bool is_collinear(Point<T> p, Point<T> q, Point<T> r) {
B84   return eq(sarea(p, q, r), (T)0);
0D1 }
```

Line

```
C97 #include "point.hpp"
72C struct Line {
2DF   Ptd p, q;
2BD   Line() {}
299   Line(Ptd p, Ptd q) : p(p), q(q) {}
F5B };
    // se p pertence ao segmento r
7E1 bool isinseg(Ptd p, Line r) {
AAC   Ptd a = r.p - p, b = r.q - p;
B7E   return eq(a.cross(b), 0) && a.dot(b) < eps;
C3B }
    // projecao do ponto p na reta r
15B Ptd proj(Ptd p, Line r) {
BEA   if (r.p == r.q) return r.p;
2FF   Ptd v = r.q - r.p, u = p - r.p;
5C6   return r.p + v * (u.dot(v) / v.dot(v));
2E8 }
    // Intersecao das retas; retorna {INF, INF} se paralelas.
24B Ptd inter(Line r, Line s) {
CAD   Ptd u = r.q - r.p, v = s.q - s.p, w = s.p - r.p;
018   ld d = u.cross(v);
F2F   if (eq(d, 0)) return {1e18, 1e18};
2C9   return r.p + u * (w.cross(v) / d);
15F }
    // se o segmento r intersecta o segmento s
090 bool interseg(Line r, Line s) {
469   if (isinseg(r.p, s) || isinseg(r.q, s) || isinseg(s.p, r) ||
C1D       isinseg(s.q, r))
8A6     return true;
D02   auto ccw = [](Ptd a, Ptd b, Ptd c) {
9B9     return (b - a).cross(c - a) > eps;
3ED   };
13C   return ccw(r.p, r.q, s.p) != ccw(r.p, r.q, s.q) &&
413          ccw(s.p, s.q, r.p) != ccw(s.p, s.q, r.q);
7D4 }
    // distancia do ponto p a reta r
E49 ld disttoline(Ptd p, Line r) {
FF0   return abs((r.q - r.p).cross(p - r.p)) / (r.q - r.p).len();
2F5 }
    // distancia do ponto p ao segmento r
A20 ld disttoseg(Ptd p, Line r) {
5A8   if ((r.q - r.p).dot(p - r.p) < 0) return (p - r.p).len();
A6E   if ((r.p - r.q).dot(p - r.q) < 0) return (p - r.q).len();
A19   return disttoline(p, r);
9C0 }
    // distancia entre dois segmentos
68D ld distseg(Line a, Line b) {
4DF   if (interseg(a, b)) return 0;
3C3   return min({disttoseg(a.p, b), disttoseg(a.q, b),
458               disttoseg(b.p, a), disttoseg(b.q, a)});
A44 }
```

Circle

```
5FC #include "line.hpp"
    // centro da circunferencia que passa por a, b, c
880 Ptd getcenter(Ptd a, Ptd b, Ptd c) {
3FB   Ptd m1 = (a + b) / 2, m2 = (a + c) / 2;
1E3   return inter(Line(m1, m1 + (b - a).perp()),
9A7                Line(m2, m2 + (c - a).perp()));
E56 }
    // intersecao da circunferencia (c, r) com a reta ab
E6C vector<Ptd> circ_line_inter(Ptd a, Ptd b, Ptd c, ld r) {
5C2   vector<Ptd> points;
FAD   Ptd u = b - a, v = a - c;
F37   ld A = u.dot(u), B = v.dot(u), C = v.dot(v) - r * r;
1FA   ld D = B * B - A * C;
4AA   if (D < -eps) return points;
B40   points.push_back(c + v + u * (-B + sqrt(D + eps)) / A);
AFA   if (D > eps) points.push_back(c + v + u * (-B - sqrt(D)) / A);
97E   return points;
F33 }
    // intersecao das circunferencias (a, r) e (b, R)
24A vector<Ptd> circ_inter(Ptd a, Ptd b, ld r, ld R) {
5C2   vector<Ptd> points;
8C7   ld d = (b - a).len();
701   if (d > r + R + eps || d + min(r, R) < max(r, R) - eps)
97E     return points;
398   ld x = (d * d - R * R + r * r) / (2 * d);
A5C   ld y = sqrt(max(0.0, r * r - x * x));
223   Ptd v = (b - a) / d;
EED   points.push_back(a + v * x + v.perp() * y);
526   if (y > eps) points.push_back(a + v * x - v.perp() * y);
97E   return points;
36B }
```

Polygon

```
5FC #include "line.hpp"
    // 2x a area com sinal (>0 = CCW)
67A template <typename T>
CB6 T area2(const vector<Point<T>> &poly) {
D0C   T a = 0;
BC6   int n = poly.size();
830   for (int i = 0; i < n; i++)
F7A     a += poly[i].cross(poly[(i + 1) % n]);
3F5   return a;
1B9 }
    // area do poligono
67A template <typename T>
402 ld polarea(const vector<Point<T>> &poly) {
323   return abs((ld)area2(poly)) / 2.0;
3C3 }
    // 0 = fora, 1 = dentro, 2 = na borda
    // O(n) para qualquer poligono simples, convexo ou concavo.
67A template <typename T>
EA9 int winding(const vector<Point<T>> &poly, Point<T> p) {
C24   int n = poly.size(), w = 0;
603   for (int i = 0; i < n; i++) {
D79     Point<T> a = poly[i] - p, b = poly[(i + 1) % n] - p;
972     if (a.y <= 0 && b.y > 0) {
E31       T c = a.cross(b);
346       if (c > 0) w++;
EAB       else if (c == 0) return 2;
305     } else if (b.y <= 0 && a.y > 0) {
E31       T c = a.cross(b);
6EA       if (c < 0) w--;
EAB       else if (c == 0) return 2;
992     } else if (b.y == 0 && a.y == 0) {
09D       if (min(a.x, b.x) <= 0 && 0 <= max(a.x, b.x)) return 2;
88E     }
C79   }
2EA   return w != 0 ? 1 : 0;
E31 }
    // Ponto no casco convexo CCW sem colineares — O(log n).
67A template <typename T>
406 bool inside_hull(const vector<Point<T>> &hull, Point<T> p) {
ACE   int n = hull.size();
D3E   if (n == 1) return p == hull[0];
4E1   if (n == 2)
DA7     return hull[0].cross(hull[1], p) == 0 &&
540            (p - hull[0]).dot(hull[1] - hull[0]) >= 0 &&
F0B            (p - hull[1]).dot(hull[0] - hull[1]) >= 0;
D16   if (hull[0].cross(hull[1], p) <= 0 ||
095       hull[0].cross(hull[n - 1], p) >= 0)
D1F     return false;
B9E   int lo = 1, hi = n - 1;
3D1   while (lo + 1 < hi) {
C86     int mid = (lo + hi) / 2;
353     if (hull[0].cross(hull[mid], p) > 0) lo = mid;
8C0     else hi = mid;
575   }
3B9   return hull[lo].cross(hull[lo + 1], p) >= 0;
FF3 }
    // Corta pela reta r; mantem p quando (r.p, r.q, p) e CCW.
010 vector<Ptd> cut_polygon(vector<Ptd> poly, Line r) {
2D3   vector<Ptd> res;
BC6   int n = poly.size();
E89   auto ccw = [&](Ptd a, Ptd b, Ptd c) {
9B9     return (b - a).cross(c - a) > eps;
2B6   };
603   for (int i = 0; i < n; i++) {
D30     if (ccw(r.p, r.q, poly[i])) res.push_back(poly[i]);
C6F     if (n == 1) continue;
F59     Line s(poly[i], poly[(i + 1) % n]);
DEA     Ptd p = inter(r, s);
D7C     if (isinseg(p, s)) res.push_back(p);
C91   }
E88   res.erase(unique(res.begin(), res.end()), res.end());
EC2   if (res.size() > 1 && res.back() == res[0]) res.pop_back();
B50   return res;
DAD }
    // se dois poligonos se intersectam — O(n*m)
B8C bool interpol(const vector<Ptd> &v1, const vector<Ptd> &v2) {
7D1   int n = v1.size(), m = v2.size();
68F   for (auto &p : v1)
E34     if (winding(v2, p)) return true;
043   for (auto &p : v2)
240     if (winding(v1, p)) return true;
830   for (int i = 0; i < n; i++)
A75     for (int j = 0; j < m; j++)
2B6       if (interseg(Line(v1[i], v1[(i + 1) % n]),
DBE                    Line(v2[j], v2[(j + 1) % m])))
8A6         return true;
D1F   return false;
8F8 }
    // Pick: I = (area2 - boundary + 2) / 2, boundary =
    // Σgcd(|dx|,|dy|) nas arestas
5CC ll lattice_points(ll area2, ll boundary) {
4EE   return (area2 - boundary + 2) / 2;
4FB }
    // distancia entre dois poligonos
CCD ld distpol(const vector<Ptd> &v1, const vector<Ptd> &v2) {
F6B   if (interpol(v1, v2)) return 0;
7D1   int n = v1.size(), m = v2.size();
D63   ld ret = 1e18;
830   for (int i = 0; i < n; i++)
A75     for (int j = 0; j < m; j++)
7DB       ret = min(ret, distseg(Line(v1[i], v1[(i + 1) % n]),
D87                              Line(v2[j], v2[(j + 1) % m])));
EDF   return ret;
B9D }
```

Convex Hull

```
046 #include "polygon.hpp"
    // Casco convexo CCW, O(N log N); descarta colineares.
67A template <typename T>
580 vector<Point<T>> convex_hull(vector<Point<T>> pts) {
CA0   int n = pts.size();
3C9   if (n < 2) return pts;
CD7   sort(pts.begin(), pts.end());
9C8   vector<Point<T>> h;
```

```
721   h.reserve(2 * n);
107   for (int phase = 0; phase < 2; phase++) {
5C9     int start = h.size();
D9F     for (auto &p : pts) {
C31       while ((int)h.size() >= start + 2) {
531         auto a = h[h.size() - 2], b = h.back();
9F6         if ((b - a).cross(p - a) > 0) break;
BFB         h.pop_back();
0ED       }
A49       h.push_back(p);
37A     }
BFB     h.pop_back();
05C     reverse(pts.begin(), pts.end());
0D0   }
E30   if (h.size() == 2 && h[0] == h[1]) h.pop_back();
81C   return h;
5AD }
    // Operacoes O(log n) em casco CCW sem colineares.
6E2 struct ConvexPol {
1C4   vector<Ptd> pol;

C45   ConvexPol() {}
D90   ConvexPol(vector<Ptd> v) : pol(convex_hull(v)) {}

      // se o ponto esta dentro do casco — O(log n)
46F   bool is_inside(Ptd p) {
B1C     int n = pol.size();
03D     if (n == 0) return false;
E08     return inside_hull(pol, p);
66C   }

      // cmp(p, q) indica que p e mais extremo que q.
8AA   int extreme(const function<bool(Ptd, Ptd)> &cmp) {
B1C     int n = pol.size();
4A2     auto extr = [&](int i, bool &cur_dir) {
22A       cur_dir = cmp(pol[(i + 1) % n], pol[i]);
35F       return !cur_dir && !cmp(pol[(i + n - 1) % n], pol[i]);
DDB     };
63D     bool last_dir, cur_dir;
A0D     if (extr(0, last_dir)) return 0;
993     int l = 0, r = n;
EAD     while (l + 1 < r) {
EE4       int m = (l + r) / 2;
F29       if (extr(m, cur_dir)) return m;
44A       bool rel_dir = cmp(pol[m], pol[l]);
72A       if ((!last_dir && cur_dir) ||
D60           (last_dir == cur_dir && rel_dir == cur_dir)) {
8A6         l = m;
1F1         last_dir = cur_dir;
69C       } else r = m;
D0E     }
792     return l;
B61   }

      // Índice do ponto com maior produto interno com v — O(log n).
82A   int max_dot(Ptd v) {
52C     return extreme(
30C       [&](Ptd p, Ptd q) { return p.dot(v) > q.dot(v); });
C6D   }

      // Tangentes por p: indices {esquerda, direita} — O(log n).
D16   pair<int, int> tangents(Ptd p) {
2B1     auto left = [&](Ptd q, Ptd r) {
540       return (r - p).cross(q - p) > eps;
3F4     };
183     auto right = [&](Ptd q, Ptd r) {
F28       return (q - p).cross(r - p) > eps;
0FA     };
8EB     return {extreme(left), extreme(right)};
1AB   }
A34 };
```

Closest Pair

```
C97 #include "point.hpp"

    /* Closest Pair — Par de pontos mais proximos
     * Complexidade: O(N log N)
     *
     * closest_pair(pts) → indices {i, j} do par; i < j.
     *
     * Distancia ao quadrado: (pts[i] - pts[j]).dist2()
     * Funciona com coordenadas inteiras, sem erro de precisao.
     */
67A template <typename T>
CCC pair<int, int> closest_pair(const vector<Point<T>> &pts) {
CA0   int n = pts.size();
135   vector<int> idx(n);
295   iota(idx.begin(), idx.end(), 0);
658   sort(idx.begin(), idx.end(),
866        [&](int a, int b) { return pts[a].x < pts[b].x; });
3C9   T best = (pts[idx[0]] - pts[idx[1]]).dist2();
D94   pair<int, int> ans = {idx[0], idx[1]};
      // set ordenado por (y, indice)
3B1   set<pair<T, int>> active;
637   for (int l = 0, r = 0; r < n; r++) {
BD5     const Point<T> &pr = pts[idx[r]];
FAF     T delta = (T)ceil(sqrt((double)best)) + 1;
3D6     while (pr.x - pts[idx[l]].x > delta) {
9CB       active.erase({pts[idx[l]].y, idx[l]});
63B       l++;
34B       delta = (T)ceil(sqrt((double)best)) + 1;
B99     }
4A9     auto lo = active.lower_bound({pr.y - delta, INT_MIN});
A87     auto hi = active.upper_bound({pr.y + delta, INT_MAX});
DC0     for (auto it = lo; it != hi; it++) {
6BD       T d = (pr - pts[it->second]).dist2();
1D6       if (d < best) {
365         best = d;
5A5         ans = {idx[r], it->second};
C22       }
D09     }
D2B     active.insert({pr.y, idx[r]});
FB8   }
591   if (ans.first > ans.second) swap(ans.first, ans.second);
BA7   return ans;
A71 }
```

Others

Stress Test

```
P=code   # codigo a testar
Q=brute  # brute correto
C=       # checker; vazio usa cmp

g++ "$P.cpp" -o sol -O2 || exit 1
g++ "$Q.cpp" -o brt -O2 || exit 1
g++ gen.cpp -o gen -O2 || exit 1
if [ -n "$C" ]; then
  g++ "$C.cpp" -o chk -O2 || exit 1
fi

for ((i = 1; ; i++)); do
  echo "$i"
  ./gen "$i" > in
  ./sol < in > out
  ./brt < in > out2
  if [ -n "$C" ]; then
    ./chk in out out2
  else
    cmp -s out out2
  fi
  if [ $? -ne 0 ]; then
    echo "--> entrada:"
    cat in
    echo "--> saida code:"
    cat out
    echo "--> saida brute:"
    cat out2
    break
  fi
done
```

Gen

```
42A #define endl '\n'

E72 auto seed =
415   chrono::steady_clock::now().time_since_epoch().count();
F1A mt19937 rng(seed);
    // int x = rng();
463 int uniform(int l, int r) {
A7F   uniform_int_distribution<int> uid(l, r);
F54   return uid(rng);
D9E }
E8D int main() {
886   ios::sync_with_stdio(false);
B95   cin.tie(0);
BB3   return 0;
268 }
```

Checker

```
F3B int main(int argc, char *argv[]) {
4DE   ifstream in(argv[1]), out_sol(argv[2]), out_brt(argv[3]);
BB3   return 0; // 0 = AC, 1 = WA
231 }
```

Hash Function

O hash de cada subseção é calculado com MD5 (3 chars) sobre o código sem comentários e sem espaços em branco. Usado para verificar integridade durante contest.
O hash de uma linha com } é o hash de todo o código desde a { que a abriu.
Para verificar: copie o código, rode o verificador e compare o prefixo.

Hash Verifier

```
95B string getHash(string code) {
049   ofstream("z.cpp") << code;
FC0   system("g++ -E -P -dD -fpreprocessed ./z.cpp"
041          " | tr -d '[:space:]' | md5sum > sh");
DB3   ifstream("sh") >> code;
78A   return code.substr(0, 3);
497 }
E8D int main() {
E82   string line, context;
0EE   stack<string> blocks({""});
266   while (getline(cin, line)) {
063     context = line;
808     for (char c : line)
66E       if (c == '{') blocks.push("");
CF0       else if (c == '}')
19B         context = blocks.top() + line, blocks.pop();
661     cout << getHash(context) + " " + line << endl;
4F5     blocks.top() += context;
697   }
56A }
```

Tabelas de Referência

Tipos, Constantes & Complexidade

type	bits	max	fmt
int	32	$\approx 2.1\times 10^9$	%d
uint	32	$\approx 4.3\times 10^9$	%u
ll	64	$\approx 9.2\times 10^{18}$	%lld
ull	64	$\approx 1.8\times 10^{19}$	%llu
double	64	10^{308}, $\varepsilon\approx 10^{-15}$	%lf
long dbl	80	10^{4932}, $\varepsilon\approx 10^{-18}$	%Lf

valor	obs
$12! = 479001600$	int
$20! = 2.4\times 10^{18}$	ll
$\binom{33}{16}\approx 1.2\times 10^9$	int
$\binom{66}{33}\approx 7.2\times 10^{18}$	ll

$2^{-1} \bmod p$	
10^9+7	500000004
10^9+9	500000005
998244353	499122177
$2^{61}-1$	1152921504606846976

n	$\pi(n)$
10^4	1229
10^5	9592
10^6	78498
10^7	664579
10^8	5761455
10^9	50847534

n	O viavel
10^8	$O(n)$
10^7	$O(n\log n)$
5×10^3	$O(n^2)$
500	$O(n^3)$
22	$O(2^n n)$
11	$O(n!)$

Combinatória

Somatórios

$$\sum_{i=1}^{n} i = \frac{n(n+1)}{2},\quad \sum_{i=1}^{n} i^2 = \frac{n(n+1)(2n+1)}{6},\quad \sum_{i=1}^{n} i^3 = n^2\frac{(n+1)^2}{4}$$

$$\sum_{i=1}^{n} i^m = \frac{1}{m+1}\sum_{k=0}^{m}\binom{m+1}{k}B_k n^{m+1-k}$$

$$\sum_{i=0}^{n} c^i = \frac{c^{n+1}-1}{c-1},\quad \sum_{i=0}^{\infty} c^i = \frac{1}{1-c},\quad |c|<1$$

$$\sum_{i=0}^{n} ic^i = \frac{nc^{n+2}-(n+1)c^{n+1}+c}{(c-1)^2},\quad \sum_{i=0}^{\infty} ic^i = \frac{c}{(1-c)^2},\quad |c|<1$$

$$H_n = \sum_{i=1}^{n}\frac{1}{i} \approx \ln n + 0.5772,\quad \ln n < H_n < \ln n + 1$$

$$\sum_{i=1}^{n} H_i = (n+1)H_n - n,\quad \sum_{i=1}^{n} iH_i = \frac{n(n+1)}{2}H_n - \frac{n(n-1)}{4}$$

Identidades Binomiais

$$\binom{n}{k} = \frac{n!}{(n-k)!k!} = \binom{n}{n-k} = \frac{n}{k}\binom{n-1}{k-1} = \binom{n-1}{k} + \binom{n-1}{k-1}$$

$$\binom{n}{m}\binom{m}{k} = \binom{n}{k}\binom{n-k}{m-k},\quad \binom{n}{k} = (-1)^k\binom{k-n-1}{k}$$

$$(x+y)^n = \sum_{k=0}^{n}\binom{n}{k}x^{n-k}y^k,\quad x^n - y^n = (x-y)\sum_{k=0}^{n-1}x^{n-1-k}y^k$$

$$\sum_{k=0}^{n}\binom{n}{k} = 2^n,\quad \sum_{k=0}^{n}(-1)^k\binom{n}{k} = 0$$

$$\sum_{k=0}^{n} k\binom{n}{k} = n2^{n-1},\quad \sum_{k=0}^{n} k^2\binom{n}{k} = n(n+1)2^{n-2}$$

$$\sum_{k=0}^{n}\binom{r+k}{k} = \binom{r+n+1}{n},\quad \sum_{k=0}^{n}\binom{k}{m} = \binom{n+1}{m+1}$$

$$\sum_{k=0}^{n}\binom{r}{k}\binom{s}{n-k} = \binom{r+s}{n},\quad \sum_{k=0}^{n}\binom{n}{k}^2 = \binom{2n}{n}$$

$$\sum_{k=0}^{r}(-1)^k\binom{n}{k} = (-1)^r\binom{n-1}{r}$$

$$(1+x)^n = \sum_{k=0}^{n}\binom{n}{k}x^k,\quad (1+x)^{-n} = \sum_{k=0}^{\infty}(-1)^k\binom{n+k-1}{k}x^k$$

Inversao binomial: $f(n) = \sum_k \binom{n}{k}g(k) \Leftrightarrow g(n) = \sum_k (-1)^{n-k}\binom{n}{k}f(k)$

Contagem

Permutacoes com repeticoes $n_1,\dots,n_r$: $\frac{n!}{n_1!\cdots n_r!}$
Permutacoes circulares: $(n-1)!$
Stars & Bars: $x_1+\dots+x_k = n$, $x_i \ge 0$: $\binom{n+k-1}{k-1}$
Inclusao-Exclusao: $|\bigcup_{i=1}^{n} A_i| = \sum|A_i| - \sum|A_i\cap A_j| + \dots \pm |A_1\cap\dots\cap A_n|$

Sequências Especiais

Catalan: $C_n = \frac{1}{n+1}\binom{2n}{n}$; $1, 1, 2, 5, 14, 42, 132, 429, \dots$
Conta: arvores binarias com n nos, triangulacoes de $(n+2)$-gono, n pares de parenteses validos.

Fibonacci: $F_0 = F_1 = 1$, $F_{-i} = (-1)^{i-1}F_i$

$$F_n = \frac{\varphi^n - \psi^n}{\sqrt{5}},\quad \varphi = \frac{1+\sqrt{5}}{2},\quad \psi = \frac{1-\sqrt{5}}{2}$$

$$F_{i+1}F_{i-1} - F_i^2 = (-1)^i \quad \text{(Cassini)}$$

$$F_{n+k} = F_kF_{n+1} + F_{k-1}F_n,\quad F_{2n} = F_nF_{n+1} + F_{n-1}F_n$$

$$\gcd(F_m, F_n) = F_{\gcd(m,n)},\quad \sum_{k=1}^{n} F_k = F_{n+2} - 1,\quad \sum_{k=1}^{n} F_k^2 = F_nF_{n+1}$$

$$\begin{pmatrix} F_n & F_{n-1} \\ F_{n-1} & F_{n-2} \end{pmatrix} = \begin{pmatrix} 1 & 1 \\ 1 & 0 \end{pmatrix}^n$$

Representacao de Fibonacci: todo n tem representacao unica $n = F_{k_1} + \dots + F_{k_m}$, $k_i \ge k_{i+1}+2$, $k_m \ge 2$.

Stirling 1ª especie $\begin{bmatrix} n \\ k \end{bmatrix}$: permutacoes de n com k ciclos.

$$\begin{bmatrix} n \\ 1 \end{bmatrix} = (n-1)!,\quad \begin{bmatrix} n \\ 2 \end{bmatrix} = (n-1)!H_{n-1},\quad \begin{bmatrix} n \\ n \end{bmatrix} = 1$$

$$\begin{bmatrix} n \\ k \end{bmatrix} = (n-1)\begin{bmatrix} n-1 \\ k \end{bmatrix} + \begin{bmatrix} n-1 \\ k-1 \end{bmatrix},\quad \sum_k \begin{bmatrix} n \\ k \end{bmatrix} = n!$$

$$x^{\overline{n}} = \sum_{k=0}^{n}\begin{bmatrix} n \\ k \end{bmatrix}x^k,\quad \begin{bmatrix} n \\ n-1 \end{bmatrix} = \binom{n}{2}$$

Stirling 2ª especie $\begin{Bmatrix} n \\ k \end{Bmatrix}$: particoes de n em k subconjuntos nao vazios.

$$\begin{Bmatrix} n \\ 1 \end{Bmatrix} = \begin{Bmatrix} n \\ n \end{Bmatrix} = 1,\quad \begin{Bmatrix} n \\ 2 \end{Bmatrix} = 2^{n-1} - 1,\quad \begin{Bmatrix} n \\ n-1 \end{Bmatrix} = \begin{bmatrix} n \\ n-1 \end{bmatrix} = \binom{n}{2}$$

$$\begin{Bmatrix} n \\ k \end{Bmatrix} = k\begin{Bmatrix} n-1 \\ k \end{Bmatrix} + \begin{Bmatrix} n-1 \\ k-1 \end{Bmatrix} = \frac{1}{k!}\sum_{j=0}^{k}(-1)^{k-j}\binom{k}{j}j^n$$

Bell $B_n = \sum_k \begin{Bmatrix} n \\ k \end{Bmatrix}$: # particoes de $\{1,\dots,n\}$ em subconjuntos nao vazios.
$B_0 = 1, B_1 = 1, B_2 = 2, B_3 = 5, B_4 = 15, B_5 = 52, B_6 = 203$

Numeros de Euler $\left\langle {n \atop k} \right\rangle$: permutacoes de n com k ascendentes.

$$\left\langle {n \atop 0} \right\rangle = \left\langle {n \atop n-1} \right\rangle = 1,\quad \left\langle {n \atop k} \right\rangle = \left\langle {n \atop n-1-k} \right\rangle,\quad \left\langle {n \atop k} \right\rangle = (k+1)\left\langle {n-1 \atop k} \right\rangle + (n-k)\left\langle {n-1 \atop k-1} \right\rangle$$

$$\left\langle {n \atop 1} \right\rangle = 2^n - n - 1,\quad \left\langle {n \atop m} \right\rangle = \sum_{k=0}^{m}(-1)^k\binom{n+1}{k}(m+1-k)^n$$

$$x^n = \sum_{k=0}^{n}\left\langle {n \atop k} \right\rangle\binom{x+k}{n}$$

Derangements: $D_n = (n-1)(D_{n-1} + D_{n-2})$; $D_n \approx \frac{n!}{e}$

Josephus: $f(1,k) = 0$, $f(n,k) = (f(n-1,k) + k) \bmod n$

Funções Geradoras

Serie ordinaria: $A(x) = \sum_{i\ge 0} a_i x^i$; exponencial: $A(x) = \sum_{i\ge 0} a_i \frac{x^i}{i!}$

$$\frac{1}{1-x} = \sum x^i,\quad e^x = \sum\frac{x^i}{i!},\quad \ln(1+x) = \sum(-1)^{i+1}\frac{x^i}{i}$$

$$(1+x)^n = \sum\binom{n}{i}x^i,\quad \frac{x}{1-x-x^2} = \sum F_i x^i$$

$$\frac{1}{2x}\left(1 - \sqrt{1-4x}\right) = \sum C_i x^i,\quad \frac{1}{\sqrt{1-4x}} = \sum\binom{2i}{i}x^i$$

Convolucao: $A(x)B(x) = \sum_i\left(\sum_{j=0}^{i} a_j b_{i-j}\right)x^i$.
Soma acumulada: $B(x) = \frac{1}{1-x}A(x)$ se $b_i = \sum_{j\le i} a_j$.

Potências Fatoriais & Bernoulli

$$x^{\underline{n}} = x(x-1)\cdots(x-n+1),\quad x^{\overline{n}} = x(x+1)\cdots(x+n-1)$$

$$x^{\underline{n}} = \sum_{k=1}^{n}\begin{bmatrix} n \\ k \end{bmatrix}(-1)^{n-k}x^k,\quad x^{\overline{n}} = \sum_{k=1}^{n}\begin{bmatrix} n \\ k \end{bmatrix}x^k$$

$$x^n = \sum_{k=1}^{n}\begin{Bmatrix} n \\ k \end{Bmatrix}x^{\underline{k}}$$

Numeros de Bernoulli ($B_i = 0$ para i impar >1):
$B_0 = 1$, $B_1 = -\frac{1}{2}$, $B_2 = \frac{1}{6}$, $B_4 = -\frac{1}{30}$, $B_6 = \frac{1}{42}$, $B_8 = -\frac{1}{30}$

$$\frac{x}{e^x - 1} = \sum_{i\ge 0} B_i\frac{x^i}{i!},\quad n! = \sqrt{2\pi n}\left(\frac{n}{e}\right)^n\left(1 + \Theta\left(\frac{1}{n}\right)\right)$$

Teoria dos Números

Euler Phi & Möbius

$\varphi(n)$: quantidade de inteiros em $[1,n]$ coprimos com n.

$$\varphi(n) = n\prod_{p\,|\,n}\left(1 - \frac{1}{p}\right) = \prod_{i=1}^{n} p_i^{e_i-1}(p_i - 1)$$

$$\varphi(mn) = \varphi(m)\varphi(n)\ (\gcd = 1),\quad \sum_{d\,|\,n}\varphi(d) = n$$

$$a^{\varphi(m)} \equiv 1 \pmod m$$

$$\mu(n) = \begin{cases} 1 & n = 1 \\ (-1)^r & n = p_1\cdots p_r, \\ 0 & p^2 \mid n \end{cases}\quad \sum_{d\,|\,n}\mu(d) = [n = 1]$$

Inversão de Möbius: $g(n) = \sum_{d|n} f(d) \Leftrightarrow f(n) = \sum_{d|n}\mu\left(\frac{n}{d}\right)g(d)$

$$\frac{1}{\zeta(x)} = \sum_{i=1}^{\infty}\frac{\mu(i)}{i^x},\quad \frac{\zeta(x-1)}{\zeta(x)} = \sum_{i=1}^{\infty}\frac{\varphi(i)}{i^x},\quad \zeta^2(x) = \sum_{i=1}^{\infty}\frac{d(i)}{i^x}$$

GCD, Diofantinas & TCR

$\gcd(a,b) = \gcd(b, a \bmod b)$; $\operatorname{lcm}(a,b) = ab/\gcd(a,b)$
Bezout: $ax + by = \gcd(a,b)$ sempre tem solução inteira. Em geral, $ax + by = c$ tem solução $\Leftrightarrow \gcd(a,b) \mid c$.
TCR: Sistema $x \equiv a_i \pmod{m_i}$ com m_i coprimos dois a dois tem solucao unica $\bmod M = \prod m_i$:

$$x = \sum_i a_i M_i (M_i^{-1} \bmod m_i),\quad M_i = \frac{M}{m_i}$$

Aritmetica Modular

Inverso modular: $a^{-1} \bmod m$ via $a^{m-2} \bmod m$ (requer m primo). Para todos os inversos $1\dots n$ de uma vez:

$$\operatorname{inv}[1] = 1,\quad \operatorname{inv}[i] = -(m/i)\cdot\operatorname{inv}[m \bmod i] \bmod m$$

Lucas: Para p primo, $\binom{n}{k} \bmod p = \binom{n \bmod p}{k \bmod p}\cdot\binom{\lfloor \frac{n}{p}\rfloor}{\lfloor \frac{k}{p}\rfloor} \pmod p$

Recorrências & Séries

Teorema Mestre

$T(n) = aT\left(\frac{n}{b}\right) + f(n)$, $a \ge 1$, $b > 1$:
$f(n) = O\left(n^{\log_b a - \varepsilon}\right) \Rightarrow T(n) = \Theta\left(n^{\log_b a}\right)$
$f(n) = \Theta\left(n^{\log_b a}\right) \Rightarrow T(n) = \Theta\left(n^{\log_b a}\log n\right)$
$f(n) = \Omega\left(n^{\log_b a + \varepsilon}\right)$ e $af\left(\frac{n}{b}\right) \le cf(n) \Rightarrow T(n) = \Theta(f(n))$

Séries de Taylor

$$e^x = \sum\frac{x^i}{i!},\quad \ln(1+x) = \sum(-1)^{i+1}\frac{x^i}{i},\quad \ln\frac{1}{1-x} = \sum\frac{x^i}{i}$$

$$\sin x = \sum(-1)^i\frac{x^{2i+1}}{(2i+1)!},\quad \cos x = \sum(-1)^i\frac{x^{2i}}{(2i)!}$$

$$\arctan x = \sum(-1)^i\frac{x^{2i+1}}{2i+1},\quad \frac{1}{(1-x)^{n+1}} = \sum\binom{i+n}{i}x^i$$

Teoria dos Grafos

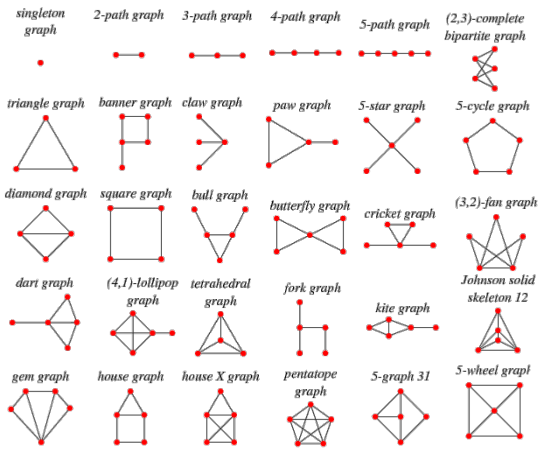
Teoremas

Euler: Circuito Euleriano $\Leftrightarrow$ todos os vertices tem grau par. Caminho Euleriano $\Leftrightarrow$ exatamente 2 vertices de grau impar.
Fórmula de Euler (planar): $V - E + F = 2$. Grafo planar simples: $E \le 3V - 6$; planar bipartido: $E \le 2V - 4$. Todo planar tem algum vertice de grau ≤ 5.
Cayley: Número de árvores geradoras rotuladas de K_n e n^{n-2}.
Kirchhoff: Número de árvores geradoras = qualquer cofator da matriz Laplaciana $L = D - A$.

Konig: Em grafo bipartido: cobertura mínima de vértices = emparelhamento máximo; conjunto independente máximo $= V -$ emparelhamento máximo.
Hall: Existe emparelhamento perfeito de $U \Leftrightarrow \forall S \subseteq U : |N(S)| \geq |S|$.
Menger: Número máximo de caminhos aresta-disjuntos entre s e t = tamanho do corte mínimo s-t.
Dilworth: Em poset, partição mínima em cadeias = anti-cadeia máxima.
Mirsky: Em poset, partição mínima em anti-cadeias = comprimento da maior cadeia.
Prüfer: Bijecao entre árvores rotuladas de n vértices e sequências de comprimento $n-2$ sobre $\{1, ..., n\}$.
Erdos-Gallai: Sequência $d_1 \geq ... \geq d_n$ é sequência de graus de algum grafo simples $\Leftrightarrow \sum d_i$ e par e para todo k:

$$\sum_{i=1}^{k} d_i \leq k(k-1) + \sum_{i=k+1}^{n} \min(d_i, k)$$

Tipos de Grafo



Matemática

Probabilidade

Bayes: Atualiza a probabilidade de A dado que B ocorreu.

$$P(A|B) = \frac{P(B|A)P(A)}{P(B)}$$

Quando $P(B)$ nao e conhecida diretamente (A_j sao hipoteses mutuamente exclusivas e exaustivas):

$$P(A_i|B) = \frac{P(B|A_i)P(A_i)}{\sum_j P(B|A_j)P(A_j)}$$

Esperanca: $E[X] = \sum x P(X = x)$
Linearidade: $E[aX + bY] = aE[X] + bE[Y]$. Se independentes: $E[XY] = E[X]E[Y]$
Variancia: $\text{Var}(X) = E[X^2] - E[X]^2$; $\text{Var}(aX + b) = a^2 \text{ Var}(X)$
Se independentes: $\text{Var}(X + Y) = \text{Var}(X) + \text{Var}(Y)$
Markov: $P[X \geq \lambda E[X]] \leq \frac{1}{\lambda}$
Chebyshev: $P[|X - E[X]| \geq \lambda\sigma] \leq \frac{1}{\lambda^2}$

Binomial $B(n,p)$: n ensaios de Bernoulli independentes com prob p.

$$P(X = k) = \binom{n}{k} p^k (1-p)^{n-k}, \quad E[X] = np, \quad \text{Var} = np(1-p)$$

Geometrica $\text{Geom}(p)$: numero de tentativas ate o primeiro sucesso ($q = 1 - p$).

$$P(X = k) = pq^{k-1}, \quad E[X] = \frac{1}{p}, \quad \text{Var} = \frac{q}{p^2}$$

Hipergeometrica $H(N, K, n)$: N itens, K marcados, n sorteados sem reposicao.

$$P(X = k) = \binom{K}{k} \frac{\binom{N-K}{n-k}}{\binom{N}{n}}, \quad E[X] = \frac{nK}{N}$$

Poisson $\text{Pois}(\lambda)$: aproximacao de $B(n,p)$ quando $n \to \infty$, $np = \lambda$.

$$P(X = k) = \frac{e^{-\lambda}\lambda^k}{k!}, \quad E[X] = \text{Var}(X) = \lambda$$

Normal $\mathcal{N}(\mu, \sigma^2)$: $p(x) = \frac{1}{\sqrt{2\pi}\sigma} e^{-(x-\mu)^2/2\sigma^2}$, $E[X] = \mu$, $\text{Var} = \sigma^2$.

Coupon collector: n tipos equiprovaveis; numero esperado de sorteios $= nH_n$.
Paradoxo do aniversario: $\approx 1.18\sqrt{M}$ amostras de $[M]$ para colisao com prob $\geq 50\%$.

Trigonometria

$\sin^2 x + \cos^2 x = 1$, $1 + \tan^2 x = \sec^2 x$, $1 + \cot^2 x = \csc^2 x$
$\sin(x \pm y) = \sin x \cos y \pm \cos x \sin y$
$\cos(x \pm y) = \cos x \cos y \mp \sin x \sin y$
$\tan(x \pm y) = \frac{\tan x \pm \tan y}{1 \mp \tan x \tan y}$
$\sin 2x = 2 \sin x \cos x$, $\cos 2x = \cos^2 x - \sin^2 x = 2\cos^2 x - 1 = 1 - 2\sin^2 x$
$e^{ix} = \cos x + i \sin x$, $e^{i\pi} = -1$
Lei dos senos: $\frac{a}{\sin}A = \frac{b}{\sin}B = \frac{c}{\sin}C = 2R$
Lei dos cossenos: $c^2 = a^2 + b^2 - 2ab\cos C$
Area do triangulo (lados a,b,c; angulos A,B,C):

$$\frac{1}{2}ab\sin C, \quad \sqrt{s(s-a)(s-b)(s-c)}\ s = \frac{a+b+c}{2}, \quad \frac{c^2 \sin A \sin B}{2\sin C}$$

Circumraio: $R = ab\frac{c}{4A}$. **Inraio:** $r = \frac{A}{s}$.

Geometria

Shoelace: $A = \frac{1}{2} \left| \sum_i (x_i y_{i+1} - x_{i+1} y_i) \right|$
Pick: $A = I + \frac{B}{2} - 1$ (I = pontos interiores, B = pontos na borda)
Area por coordenadas: $\frac{1}{2} \left| \det\begin{pmatrix} x_1 - x_3 & x_2 - x_3 \\ y_1 - y_3 & y_2 - y_3 \end{pmatrix} \right|$
Esferas/circulos: $A = \pi r^2$, $V = \frac{4}{3}\pi r^3$.
Produto vetorial: $\vec{u} \times \vec{v} = (u_y v_z - u_z v_y,\ u_z v_x - u_x v_z,\ u_x v_y - u_y v_x)$
Em 2D: $u_x v_y - u_y v_x$ (> 0 anti-horario, $= 0$ colinear; modulo = area do paralelogramo)
Dist. ponto-ponto: $d = \sqrt{(x_1 - x_2)^2 + (y_1 - y_2)^2}$ (3D: inclui $(z_1 - z_2)^2$)
Dist. ponto-reta ($ax + by + c = 0$): $d = |ax_0 + by_0 + c\frac{|}{\sqrt{a^2+b^2}}$
Rotacao 2D:

$$\begin{pmatrix} x' \\ y' \end{pmatrix} = \begin{pmatrix} \cos\theta & -\sin\theta \\ \sin\theta & \cos\theta \end{pmatrix} \begin{pmatrix} x \\ y \end{pmatrix}$$

Rotacao 3D em torno de x, y, z por θ ($c = \cos\theta$, $s = \sin\theta$):

$$R_x = \begin{pmatrix} 1 & 0 & 0 \\ 0 & c & -s \\ 0 & s & c \end{pmatrix}, \quad R_y = \begin{pmatrix} c & 0 & s \\ 0 & 1 & 0 \\ -s & 0 & c \end{pmatrix}, \quad R_z = \begin{pmatrix} c & -s & 0 \\ s & c & 0 \\ 0 & 0 & 1 \end{pmatrix}$$

Teoria dos Jogos

Sprague-Grundy: Todo jogo imparcial equivale a um Nim de um unico monte. O valor de Grundy (nimber) de uma posicao u e:

$$g(u) = \text{mex}\{g(v) : v \text{ alcançável de } u\}$$

onde $\text{mex}(S)$ e o menor inteiro nao negativo fora de S.
Posição perdedora $\Leftrightarrow g(u) = 0$. Em jogo composto de partes independentes:

$$g = g_1 \oplus g_2 \oplus \cdots \oplus g_k$$

Nim: k montes de tamanhos $a_1, ..., a_k$. Posição perdedora $\Leftrightarrow a_1 \oplus \cdots \oplus a_k = 0$.
Nim misere (quem pegar o ultimo *perde*): subtraia 1 de cada pilha; se o XOR resultante $\neq 0$, a posicao original e ganhadora.
Wythoff: 2 montes (a, b), $a \leq b$. Movimentos: remover qualquer qtd de um monte, ou a mesma qtd dos dois. Posição perdedora $\Leftrightarrow a = \lfloor \varphi k \rfloor$, $b = \lfloor \varphi^2 k \rfloor$ para algum $k \geq 0$, onde $\varphi = \frac{1+\sqrt{5}}{2}$.

Tabelas Numéricas

Números Primos

2	3	5	7	11	13	17	19	23	29	31	37	41	43	47	53	59	61	67	71	73
79	83	89	97	101	103	107	109	113	127	131	137	139	149	151	157	163	167	173	179	181
191	193	197	199	211	223	227	229	233	239	241	251	257	263	269	271	277	281	283	293	307
311	313	317	331	337	347	349	353	359	367	373	379	383	389	397	401	409	419	421	431	433
439	443	449	457	461	463	467	479	487	491	499	503	509	521	523	541	547	557	563	569	571
577	587	593	599	601	607	613	617	619	631	641	643	647	653	659	661	673	677	683	691	701
709	719	727	733	739	743	751	757	761	769	773	787	797	809	811	821	823	827	829	839	853
857	859	863	877	881	883	887	907	911	919	929	937	941	947	953	967	971	977	983	991	997

Fibonacci

n	F_n	n	F_n	n	F_n	n	F_n	n	F_n	n	F_n	n	F_n
0	0	1	1	2	1	3	2	4	3	5	5	6	8
7	13	8	21	9	34	10	55	11	89	12	144	13	233
14	377	15	610	16	987	17	1597	18	2584	19	4181	20	6765

Triângulo de Pascal $\binom{n}{k}$

$n \backslash k$	0	1	2	3	4	5	6	7	8	9	10
0	1										
1	1	1									
2	1	2	1								
3	1	3	3	1							
4	1	4	6	4	1						
5	1	5	10	10	5	1					
6	1	6	15	20	15	6	1				
7	1	7	21	35	35	21	7	1			
8	1	8	28	56	70	56	28	8	1		
9	1	9	36	84	126	126	84	36	9	1	
10	1	10	45	120	210	252	210	120	45	10	1

Debugging

```
-g++ -O2 -std=c++17 -Wall -Wextra
    -fsanitize=address,undefined
    -fno-sanitize-recover=all -g
```

Antes de submeter:
- Escreva casos simples se o sample não for suficiente.
- Limite de tempo apertado? Gere casos máximos.
- Memória está ok?
- Algo pode dar overflow?
- Está submetendo o arquivo certo?

Wrong Answer:
- Imprima sua solução e outputs de debug.
- Está limpando estruturas entre casos de teste?
- O algoritmo cobre todo o range de entrada?
- Releia o enunciado completo.
- Todos os casos de borda estão corretos?
- Entendeu o problema corretamente?
- Variáveis não inicializadas?
- Overflow?
- Confundindo N/M, i/j, etc.?
- Tem certeza que o algoritmo funciona?
- Que casos especiais você não considerou?
- As funções da STL funcionam como você acha?
- Adicione `asserts` e ressubmeta.
- Rode em casos criados manualmente.
- Execute o algoritmo num caso simples à mão.
- Explique o algoritmo para um colega.
- Peça para o colega revisar seu código.
- Dê uma volta, vá ao banheiro.
- O formato de saída está correto? (espaços, newlines)
- Reescreva do zero ou peça ao colega.

Runtime Error:
- Testou todos os casos de borda localmente?
- Variáveis não inicializadas?
- Lendo/escrevendo fora do range de algum vetor?
- Algum `assert` que pode falhar?
- Divisão por zero? (`mod 0`?)
- Recursão infinita possível?
- Ponteiros ou iteradores invalidados?
- Usando memória demais?

Time Limit Exceeded:
- Tem loop infinito?
- Qual é a complexidade do algoritmo?
- Está copiando dados desnecessariamente? (use referências)
- Tamanho da entrada/saída? (considere `scanf`)
- Evite `map`/`vector`; use arrays/`unordered_map`.
- O que seus colegas acham do algoritmo?

Memory Limit Exceeded:
- Quanto de memória seu algoritmo realmente precisa?
- Está limpando estruturas entre casos de teste?